Capítulo 1

Introdução

Neste capítulo inicial, é fornecido ao leitor um contexto amplo, facilitando a interpretação dos assuntos descritos ao longo deste relatório.

A introdução organiza-se em torno do enquadramento e do contexto do projeto. Nele apresentam-se os aspetos iniciais essenciais: o problema em estudo, os objetivos e a abordagem adotada. Por fim, o capítulo explicita os contributos do projeto, o planeamento e a estrutura geral do relatório.

1.1 Enquadramento/Contexto

O presente projeto surge no âmbito da Unidade Curricular (UC) Projecto / Estágio (PESTI) da Licenciatura em Engenharia Informática do Instituto Superior de Engenharia do Porto e foi desenvolvido no escritório da *OnlyHighIQ*.

Este projeto parte de uma interpretação rigorosa do problema e aplica os conhecimentos adquiridos e as boas práticas de engenharia informática. Sintetiza o estado da arte e compara abordagens alternativas. Após esta avaliação, desenha-se e implementa-se a solução, sustentada por testes e documentação. Por fim, registam-se e apresentam-se com rigor os processos, decisões e resultados.

A *OnlyHighIQ* é uma empresa fundada recentemente, encontra-se no coração da cidade do Porto, a sua localização oferece um enquadramento dinâmico e inspirador à inovação e à colaboração. Inserida numa zona vibrante, reconhecida pela sua rica história e relevância cultural, a área garante acesso facilitado a comodidades essenciais e às principais ligações de transporte, constituindo um polo conveniente para clientes e parceiros. [1]

A missão, a visão e os valores são os pilares fundamentais para definir a identidade e o propósito de uma organização.

Tabela 1.1: Missão, Visão e Valores da OnlyHighIQ

Missão	"*...to create a dynamic community where brilliant minds collaborate to develop cutting-edge solutions and innovative platforms that transform visionary ideas into reality.*"[1]
Visão	"*To create a limitless innovation landscape where the most brilliant minds, have the tools to build the future—ensuring that progress and opportunity are distributed to everyone, everywhere.*"[1]
Valores	"*Innovation Without Borders, Collaborative Excellence, Impact with Purpose, Courage to Disrupt, Principle-Led Inte-*

	grity"[1]

A OnlyYes é o seu primeiro projeto que surge como ecossistema principal da empresa mãe, o seu foco é principalmente na criação de conteúdo pago.

Neste relatório vão ser focados aspetos da propagação de conteúdo de baixa latência, mas é importante realçar que a solução implementada passa por toda a infraestrutura. A solução apresentada neste relatório, foi apenas uma etapa entre vários processos e componentes, e decisões que levarão a OnlyYes ao lançamento.

O meu percurso na OnlyHighIQ decorreu durante cerca de quatro meses, período em que fui instruído de planear, conceber e discutir a arquitetura global da aplicação, de acordo com as boas práticas transmitidas na formação AWS Cloud Architect ministrada pela empresa, bem como com os padrões e conhecimentos que fui adquirindo ao longo do meu percurso.

1.2 Descrição do Problema

Atualmente, no mercado de criação de conteúdo encontram-se diversas barreiras técnicas para pessoas com dificuldades tecnológicas. Muitos dos criadores de conteúdo, dependem de ferramentas externas para realizar uma transmissão ao vivo, então a OnlyYes quer implementar uma rede social, onde seja possível realizar uma *livestream* com poucos cliques e pré-requisitos.

Apontou-se, também, a necessidade dos criadores de conteúdo conseguirem comunicar com os utilizadores da plataforma, pelo que foi necessário realizar um *low latency chat*.

O problema resume-se à criação de uma experiência unificada de *livestream* e comunicação em tempo real, acessível em poucos cliques, sem depender de ferramentas externas.

1.2.1 Objetivos

Para a solução a desenvolver, definiu-se os seguintes objetivos:

- Modelação e criação de escalões de acesso para controlar o acesso à transmissão: após a criação do perfil privado, são criados escalões com diferentes preços e benefícios.
- Criação de canal único por criador: provisionar um canal exclusivo a cada criador.
- Desenvolvimento da interface do criador: *dashboard* para iniciar e terminar a transmissão, escolher câmara e microfone, pré-visualizar o vídeo, definir título e descrição, editar um menu de doações, e visualizar métricas.

- Desenvolvimento da interface do utilizador: página de visualização com *player* (*play*/pausa, *mute*, volume, ecrã inteiro, seleção de qualidade), com um menu de doações definido pelo criador.
- Garantia a privacidade do Criador: capacidade de controlar o acesso à câmara e ao microfone.
- Implementação do chat em direto, permite enviar mensagens para cada utilizador durante a transmissão, incluindo o criador.

Estes foram os objetivos traçados para este projeto, que serviu para integrar o Minimum Viable Product (MVP)

1.2.2 Abordagem

Para realizar os objetivos deste projeto a empresa incutiu uma metodologia ágil, que facilita a comunicação e a resolução de problemas em equipa. A metodologia SCRUM é considerado uma *framework* de desenvolvimento de *software*.

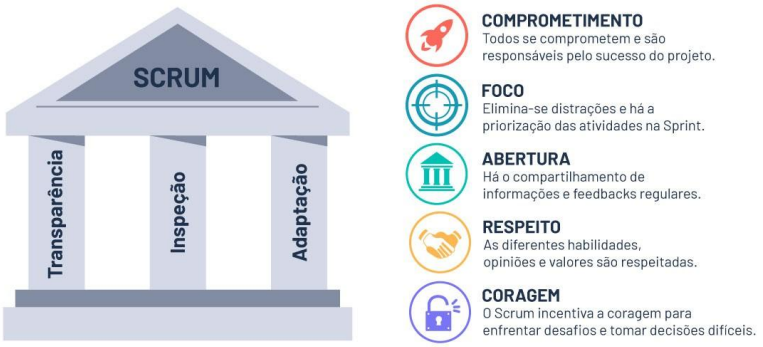

Figura 1.1: Pilares e Valores Da Metodologia SCRUM [2]

Para o projeto foi essencial seguir os pilares da metodologia: Transparecia, Reflexão (Inspeção), e Adaptação. A metodologia não serve apenas e só para a coordenação do projeto, mas os seus valores devem ser seguidos, independentemente do local de trabalho.

"Scrum is an incremental and iterative agile software development framework for managing product development (...) This strategy challenges assumptions of the "traditional, sequential approach"to product development, and encourages teams to self-organize by encouraging close online collaboration of all team members, as well as face-to-face communication among all team members and disciplines in the project.(...)"[3]

A implementação do SCRUM no projeto para além do descrito, foi feita através de *Sprints*. Os sprints são espaços de tempo definidos para mostrar as evoluções ao cliente do projeto a desenvolver. Para além disso foram realizadas *daily meetings* que permitiram verificar o processo de cada membro da equipa e avaliar o progresso do projeto como um todo.

1.2.3Contributos

Tecnológicos: Entrega de uma solução de *streaming* ligado a um controlo de acesso por *tiers*, um *player* simples e responsivo, com *chat* de baixa latência. Adoção de práticas Continuous Delivery/Continuous deployment (CI/CD) e controlo de versões.

Organizacionais. Redução da dependência de ferramentas externas e consolidação da experiência de *streaming* na própria plataforma, aumentando a capacidade de controlo e a qualidade da interação do criador com a sua comunidade.

Pedagógicos. Consolidação de competências em desenho arquitetural, integração de componentes em tempo real e entrega orientada a MVP, com documentação e avaliação objetiva do resultado.

Síntese dos contributos

- Criação de uma plataforma de *streaming* com canais exclusivos por criador.
- Controlo a canais exclusivos por *tiers* e integração com subscrições e pagamentos.
- Disponibilização de um *player* responsivo e simples (play/pausa, volume, ecrã inteiro, seleção de qualidade).
- *Chat* de baixa latência, com texto e emojis, suportado por *API WebSocket*.
- Mecanismos de privacidade e segurança
- Adoção de boas práticas de engenharia (CI/CD, controlo de versões com uso de *branching*, testes automáticos...).
- Crescimento técnico e profissional da individual e da equipa, com aplicação prática de tecnologias não abordadas em contexto académico.

1.2.4Planeamento do trabalho

Durante o estágio, o desenvolvimento da solução decorreu em etapas bem definidas. Numa fase inicial, realizou-se uma pesquisa com duração aproximada de duas semanas. Concluída essa etapa, a equipa avançou para o planeamento de Use Case (UC) e User Story (US), formalizando critérios e dependências. Seguiu-se a fase de arquitetura, durante a qual foram elaboradas as vistas lógicas, de implementação e físicas, os diagramas de sequência, o modelo de domínio e de dados, bem como a seleção das tecnologias e dos sistemas de gestão de bases de dados. Esta fase inicial estendeu-se por cerca de quatro semanas. Por fim, o supervisor organizou o trabalho em *sprints* de duas semanas no Jira, distribuindo as tarefas até à entrega da solução.

Durante os *sprints*, a equipa seguiu o formato *SCRUM* com *daily meetings*. A implementação foi organizada em fases sequenciais, a partir do desenho produzido na fase de arquitetura, criou-se a camada de domínio e a base de dados. Em seguida, desenvolveram-se as operações Create, Read, Update and Delete (CRUD) e os serviços nucleares de *backend*, prosseguiu-se com a interface de utilizador e de criador, bem como as integrações, por fim, construíram-se as funções *AWS Lambda* e efetuou-se a ligação *front-end*/*back-end*, finalizado em afinações e testes do MVP. A Tabela 1.2 sintetiza esta distribuição temporal.

1.3. Estrutura do relatório

Tabela 1.2: Divisão de tempo no projeto

Período de Tempo	Tarefa
24/02 a 07/03	Período de pesquisa
10/03 a 21/03	Fase de arquitetura
24/03 a 04/04	Sprint 1 - Criação das classes de domínio, bases de dados, e preparar o ambiente local & Inicio da pesquisa para o relatório
07/04 a 18/04	Sprint 2 - Desenvolvimento do CRUD

05/05 a 16/05	Sprint 3 - Desenvolvimento dos serviços principais de *BackEnd* & Realização dos estado da arte
19/05 a 30/05	Sprint 5 - Desenvolvimento da interface de utilizador e criador
02/06 a 13/06	Sprint 6 - Criação das lambdas e ligação entre o Front-End e Back-End
16/06 a 22/06	Sprint 7 - Afinações e testes para o MVP
23/06 a 10/09	Continuação da realização do relatório.

1.3 Estrutura do relatório

Para além da introdução, este documento contém mais 4 capítulos, que apresentam as várias etapas do desenvolvimento da solução.

No capítulo 2, é descrito o estado da arte e são apresentados trabalhos relacionados. No capítulo 3, o planeamento e o desenho, que foram previamente concebidos com a equipa para um melhor enquadramento no projeto como um todo. É realçado o domínio bem como os requesitos funcionais e não funcionais No capítulo 4, são detalhados os vários processos de implementação da solução, onde constam exemplos de funcionalidades, testes para a validação do projeto e imagens com a interface de utilizador. Por fim, o capítulo 5, concentra-se nos resultados obtidos, e numa introspeção final da solução e do relatório

Capítulo 3

Análise e desenho da solução

O capítulo que se segue tem o principal objetivo de apresentar as diferentes fases ligadas ao processo de análise e desenho da solução. Inicia-se com a contextualização do domínio do problema, suportada por um modelo de domínio e por uma tabela de termos. Procede-se à especificação dos requisitos funcionais e não funcionais, antecedendo a identificação dos atores e dos casos de uso. Por último, descreve-se o desenho da solução, evidenciando os principais componentes e a arquitetura da aplicação.

3.1 Domínio do problema

Caracteriza-se o contexto operacional da plataforma OnlyYes e o núcleo do problema a resolver descrito na secção 1.2.

Esta secção estabelece o vocabulário comum e as relações conceptuais que suportam o desenho e a implementação da solução, e o modelo que caracteriza as regras de negocio.

3.1.1 Glossário do Domínio

O glossário acrescenta significado ao domínio, pois contêm termos fundamentais para clarificar retirar ambiguidades.

A Tabela 3.1 apresenta o termo do domínio em inglês e a sua tradução em portuguesa para melhor compreensão, na última coluna da tabela são explicados os termos no contexto do domínio

Tabela 3.1: Glossário do domínio da aplicação

Termo (EN)	Termo (PT)	Descrição
Account	Conta	Entidade que representa o utilizador
Profile	Perfil	Entidade que representa a identidade do utilizador na aplicação
Livestream	Transmissão em direto	Entidade que representa a emissão de vídeo e de áudio em tempo real.
Livestream Info	Parâmetros de transmissão	Entidade que representa o conjunto de parâmetros técnicos necessários para iniciar e manter a transmissão
Tier	Escalão	Nível de acesso com preço e benefícios definidos.
Live Menu	Menu da transmissão	Conjunto de ofertas ou ações disponíveis durante a transmissão.
Live Menu Item	Item de menu	Elemento individual do menu com nome, preço e disponibilidade.
Email	Email	Endereço eletrónico associado à conta.
Username	Nome de utilizador	Identificador da conta.

Status	Estado	Situação atual de uma subscrição
Title	Título	Texto curto que identifica um conteúdo.
Description	Descrição	Texto que explica ou contextualiza um conteúdo.
streamKey	Chave de transmissão	Código utilizado para autorizar o envio do sinal de transmissão.
Channel	Canal	Unidade lógica onde as transmissões são realizadas.
Ingest Endpoint	Endereço de ingestão	Endereço utilizado para enviar o sinal de transmissão para o serviço.
Playback Url	Endereço de reprodução	Endereço utilizado para visualizar o conteúdo de uma transmissão.
Perks	Benefícios	Conjunto de vantagens incluídas num escalão.

Os temos da tabela não representam todas a nomenclaturas usadas na documentação, apenas as mais específicas e que podem gerar alguma confusão.

3.1.2Modelo de Domínio

Na figura 3.1, está apresentadoaw o diagrama conceptual Unified Modeling Language (UML) que representa entidades e relações principais.

3.2. Requisitos Funcionais e não funcionais

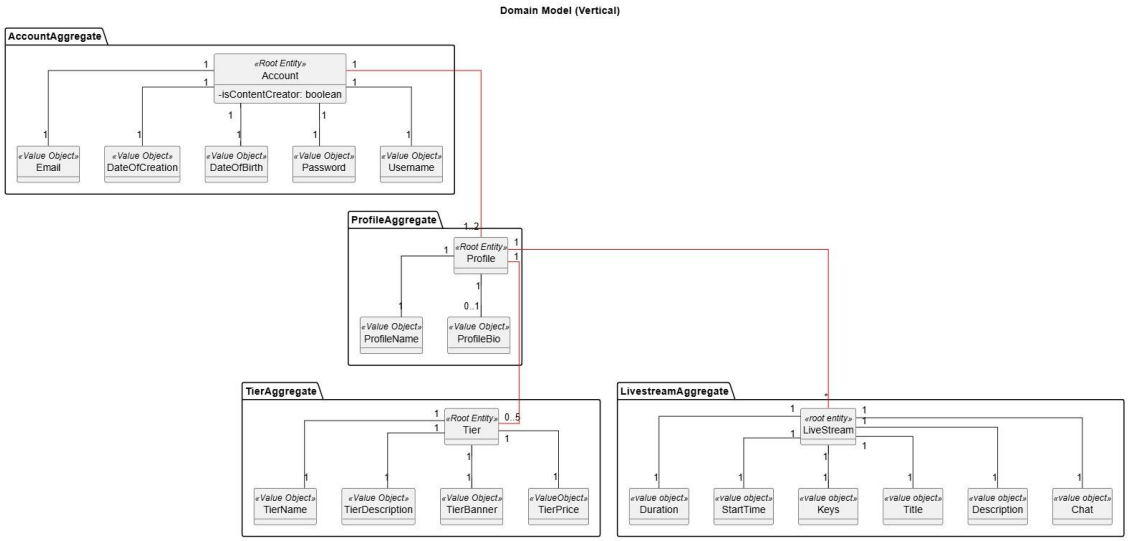

Figura 3.1: Modelo de Dominio - Streaming

A solução é composta por quatro agregados principais: *Account*, *Profile*, *Tier*, *Livestream*. Cada agregado define e representa uma área funcional do domínio, composta pela sua entidade e pelos seus objetos.

As ligações entre agregados são as seguintes:

O agregado *Account* é responsável pela integridade dos utilizadores, composta por identificadores, *email* e *username*, e por objetos que servem apenas para conformidades e para fornecer uma melhor experiência ao utilizador. O agregado *Account* liga-se ao agregado *profile*

numa ligação de um para muitos, mas no caso do domínio a sua multiplicidade será de uma para um ou dois.

O agregado *Profile* representa a identidade visual do utilizador na aplicação. Um utilizador pode ter um ou dois perfis, sendo o primeiro publico e com acesso a todos os utilizadores e o segundo só permitido a utilizadores que tenham o escalão definido pelo utilizador. Após a criação do perfil privado o sistema permite ao utilizador criar o seu ambiente de transmissão e o seu ambiente de escalões.

O agregado *Tier* modela os níveis de acesso e benefícios por perfil. Contém *TierName*, *TierDescription*, *TierBanner* e *TierPrice*, impondo as seguintes invariantes: no máximo cinco *tiers* por *Profile*.

O agregado *LiveStream* encapsula o ciclo de vida de uma transmissão em direto, incluindo *Title*, *Description*, *StartTime*, *Duration*, *Keys* e *Chat*. Este agregado é dependente da criação de um perfil privado.

3.2 Requisitos Funcionais e não funcionais

Para continuar o desenho da solução, foi necessário declarar e especificar os requisitos que a aplicação deve cumprir. Os requisitos apresentados foram definidos nas *daily meetings* com a equipa e validados pelo supervisor.

Nesta secção vão ser apresentados dois tipos de requisitos:

- Requisitos funcionais
- Requisitos não funcionais

3.2.1 Requisitos Funcionais

Um requisito funcional clarifica no âmbito funcional, as restrições e os critérios de qualidade a que a aplicação deve obedecer.

"Functional requirements capture the intended behavior of the system. This behavior may be expressed as services, tasks or functions the system is required to perform."[11]

Tabela 3.2: Requisitos Funcionais

Nº	Descrição do Requisito
RF01	O sistema deve permitir iniciar uma transmissão de forma intuitiva e rápida.
RF02	O sistema deve permitir encerrar uma transmissão de forma intuitiva e rápida.
RF03	O sistema deve bloquear o acesso a visualizadores que não possuam o *tier* adequado para visualizar uma transmissão.
RF04	O sistema deve permitir ao criador o controlo do título, descrição, câmara, microfone e cena.
RF05	O sistema deve confirmar que o criador tem perfil privado para criar uma transmissão em direto.
RF06	O sistema deve permitir que o criador personalize o menu interativo da transmissão.
RF07	O sistema deve usar o serviço de *streaming* da AWS para melhor monitorização.
RF08	O sistema deve permitir interação entre criador e visualizadores através de um *chat* em tempo real.

3.2.2 Requisitos Não Funcionais

Nesta secção estabelece-se o enquadramento dos requisitos não funcionais e do modelo FURPS+ ligando-os aos requisitos funcionais.

"Non-functional requirements help deliver a positive user experience and ensure that product performs to the standards set by stakeholders, clients, and customers."[12]

FURPS+

O modelo Functionality, Usability, Reliability, Performance,Supportability (FURPS+), serve para classificar em atributos de qualidade um softwate [12]:

- *Functionality*: Especificações funcionais que, em geral, representam as principais funcionalidades da solução.
- *Usability*: Considera aspetos da interface do utilizador, como estética e consistência.
- *Reliability*:Característica que está associada à inoperabilidade, recuperação em caso de falhas e integridade da aplicação.
- *Performance*: Refere-se aos tempos de resposta, eficiência, escalabilidade e utilização dos recursos da aplicação.
- *Supportability*: Característica relacionada com testes, compatibilidade e adaptabilidade da aplicação.

3.3. Casos de Uso

No sistema:

Functionality

- Iniciar e terminar transmissão em dois cliques (RF01/RF02)
- Autorização por escalão antes de emitir a transmissão (RF03)
- Bloquear criação da transmissão se o perfil não cumprir os critérios. (RF03/RF05)
- Controlos do criador (título, descrição, câmara, microfone, cena) com pré-visualização. (RF04)
- Menu interativo configurável pelo criador com descrição, preço e visibilidade. (RF06)
- Garantir interatividade pelo *chat* (RF08) Usability
- A interface deve ser intuitiva e consistente, permitindo ao utilizador navegar facilmente entre funcionalidades.
- O sistema deve apresentar mensagens de erro e *feedback* claro sobre o estado das operações.

Reliability

- As validações na criação e atualização de dados devem garantir a integridade da aplicação, evitando operações não autorizadas.

Performance

- O sistema deve permitir uma latência minima de acordo com a capacidade de rede de cada utilizador.

- Não deve haver perda significativa de qualidade durante a transmissão. Supportability
- A solução deve ser modular, permitindo que os componentes sejam alterados, testados e mantidos de forma independente.
- A solução deve permitir que os utilizadores utilizem diversos navegadores. + Scalability
- A estrutura modular deve permitir a integração com novas áreas de negócio sem comprometer o desempenho do sistema e a experiência do utilizador.

3.3 Casos de Uso

Na solução, foram considerados dois atores que fazem parte do sistema, o criador e o visualizador, os casos de uso foram divididos conforme as responsabilidades de cada ator.

Para melhor compreensão e legibilidade do documento os diagramas de caso de uso passaram para os anexos.A.2 A.3

Na Tabela 3.3 estão apresentados os casos de uso do criador:

Tabela 3.3: Casos de uso do ator *Creator* A.2 & A.3

ID	Caso de uso
C1	Definir um titulo para a minha transmissão
C2	Definir uma descrição para a minha transmissão
C3	Pre-Visualizar a minha transmissão
C4	Visualizar a minha transmissão
C5	Iniciar a minha transmissão
C6	Definir a minha câmara
C7	Definir o meu microfone
C8	Silenciar o meu microfone
C9	Ocultar a minha câmara
C10	Terminar a minha transmissão
C11	Visualizar o chat da minha transmissão
C12	Interagir com o chat da minha transmissão
C13	Criar um menu interativo para a minha transmissão
C14	Definir tiers para a minha transmissão

Na Tabela 3.4 estão apresentados os casos de uso do visualizador:

Tabela 3.4: Casos de uso do ator *Viewer* A.1

ID	Caso de uso
V1	Visualizar a transmissão de um Creator
V2	Interagir com o chat da transmissão
V3	Controlar a visualização [play,mute,fullscreen,return to Live]
V4	Definir a qualidade da transmissão

3.4Desenho

Nesta secção, a solução foi desenhada através do modelo C4, contempla diferentes vistas, o modelo de dados e a relação entre as tabelas. Por fim, foram desenhados os System Sequence Diagram (SSD) para completar o resto da arquitetura.

3.4.1Modelo C4

O modelo C4, apresentado por Simon Brown no vídeo [13] propõe uma abordagem hierárquica e incremental para comunicar arquiteturas de software, o modelo divise-se em quatro camadas:

- *System Context* - O sistema mais os utilizadores e as suas dependências.
- *Containers* - A forma global da arquitetura e as escolhas tecnológicas.
- *Components* - Os componentes lógicos e as respetivas interações dentro do *container*.
- *Code*- Detalhes de implementação.

3.4.2Nível 1

Na Figura 3.2 é mostrado a vista mais simples do sistema, apenas com o componente central e os serviços externos ao mesmo.

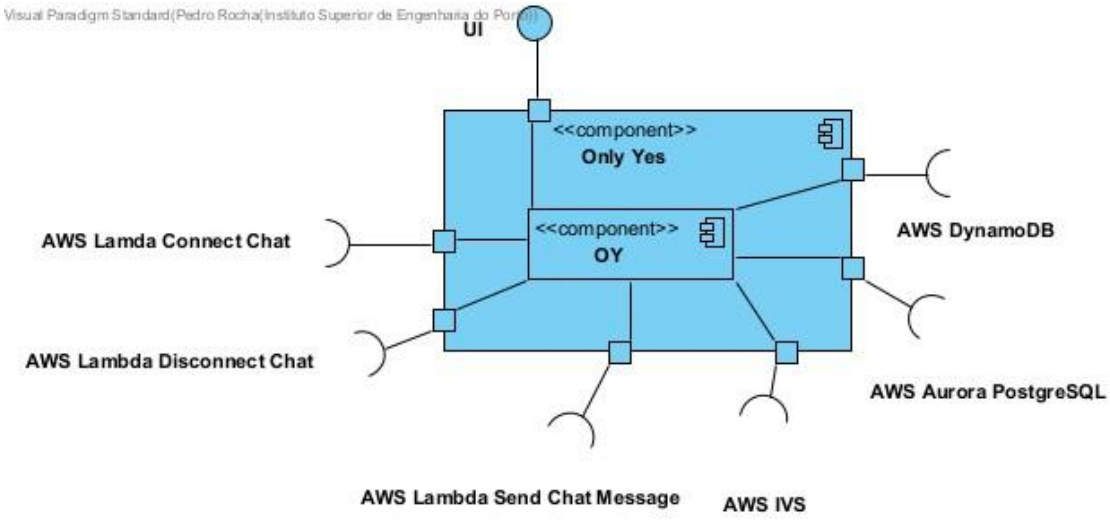

Figura 3.2: Vista Lógica do Sistema - Nível 1

O sistema disponibiliza uma interface a ser consumida pelo utilizador e consome outros serviços, descritos na Tabela 3.5 :

Tabela 3.5: Serviços disponibilizados para o consumo do sistema

Serviço	Descrição
AWS Lambda Connect Chat	Função *serverless* que permite ao utilizador juntar-se a *webSocket* para receber e enviar mensagens em tempo real.
AWS Lambda Disconect Chat	Função *serverless* que permite ao utilizador sair da *webSocket* e da sala de transmissão.
AWS Lambda Send Chat Message	Função *serverless* que permite ao utilizador enviar mensagens para o *chat* da sala de transmissão.
AWS IVS	Serviço gerido pela AWS que cria os canais de transmissão e as credenciais únicas de cada criador.
AWS Aurora PostgreSQL	Base de dados SQL gerida pela AWS.
AWS DynamoDB	Base de dados NoSQL gerida pela AWS

3.4.3 Nível 2

A Figura 3.3 apresenta-se uma vista que divide o componente principal no *front-end* e no *back-end*, cada um com as suas ligações aos respetivos serviços.

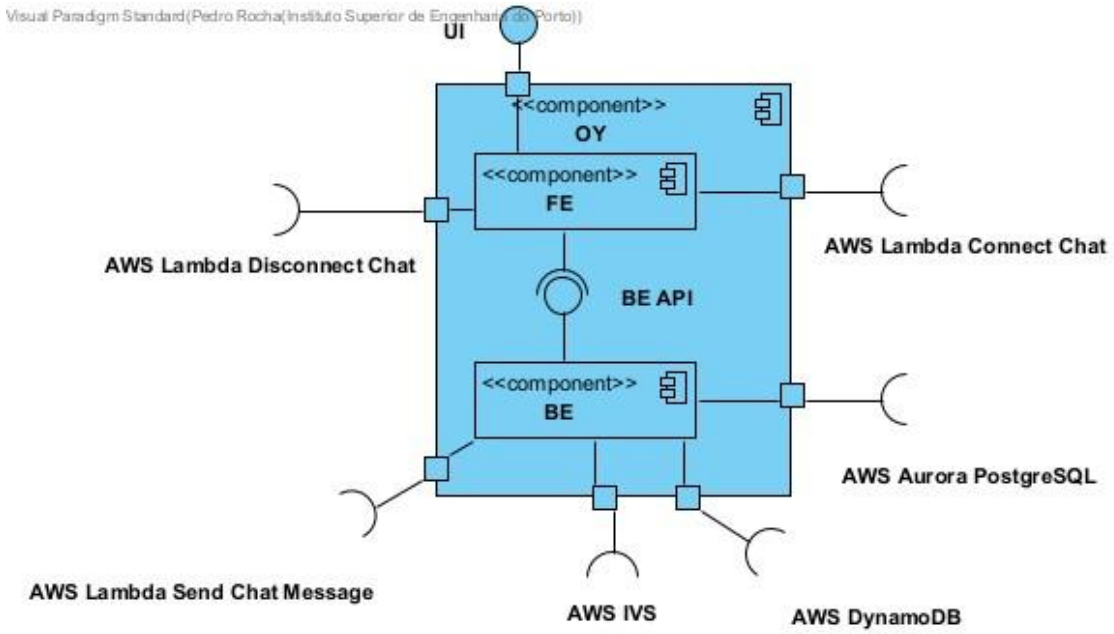

Figura 3.3: Vista Lógica do Sistema - Nível 2

O componente BE(*back-end*) proporciona uma APIREST que o componente FE(*front-end*) consome.

3.4.4Nível 3

No nível três, deste modelo, começa-se por apresentar a vista lógica do *back-end*.

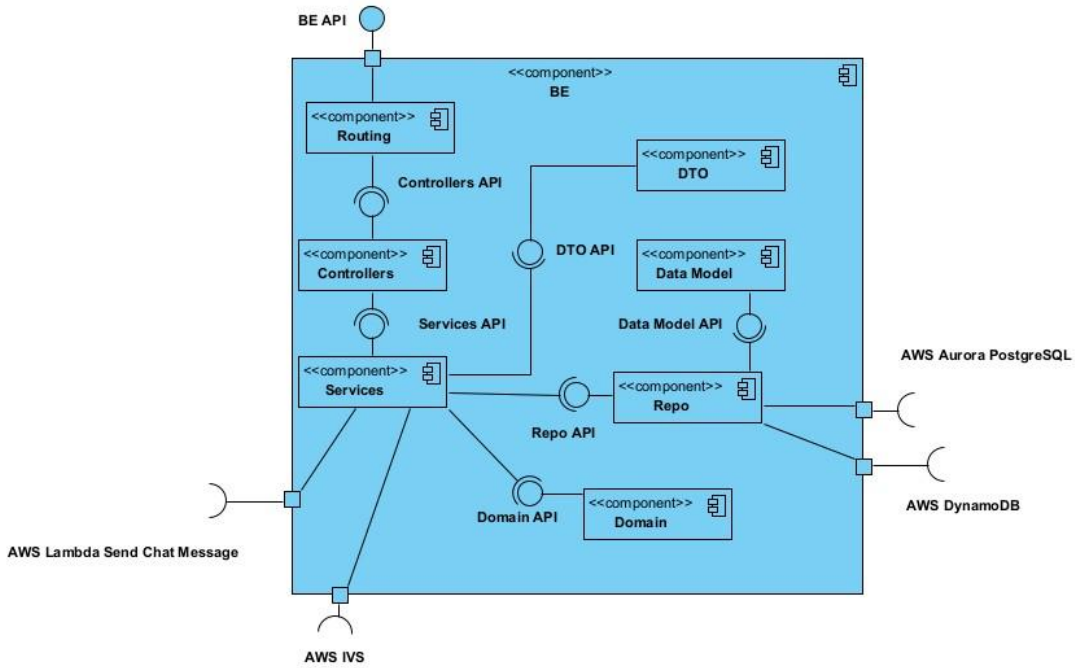

Figura 3.4: Vista Lógica do *Back-End* - Nível 3

Pela Figura 3.4 conseguimos observar as boas práticas do desenvolvimento de software, especificamente a separação de responsabilidades.

Os componentes desta vista são:

- Routing - Definem os *endpoints* da aplicação que o *front-ent* pode usar, e redireciona para os *Controllers*.
- Controllers - Estes recebem os pedidos e redirecionam para os diversos serviços da aplicação.
- Services - Contém a lógica da aplicação, servem de ponte para os repositórios.
- Data Transfer Object (DTO) - Este componente define o que tranferir de cada objeto pela aplicação.
- Data Model - Modelo de ligação ás bases de dados.
- Repo - Responsável pela persistência dos dados.
- Domain - Núcleo da aplicação, trazem o negócio para o código.

Também é possível perceber que o *back-end* integra na totalidade com as bases de dados e com os serviços que precisam de mais segurança como o IVS e a função de enviar mensagem pelo *chat*

Neste nível, também é apresentado a vista de implementação:

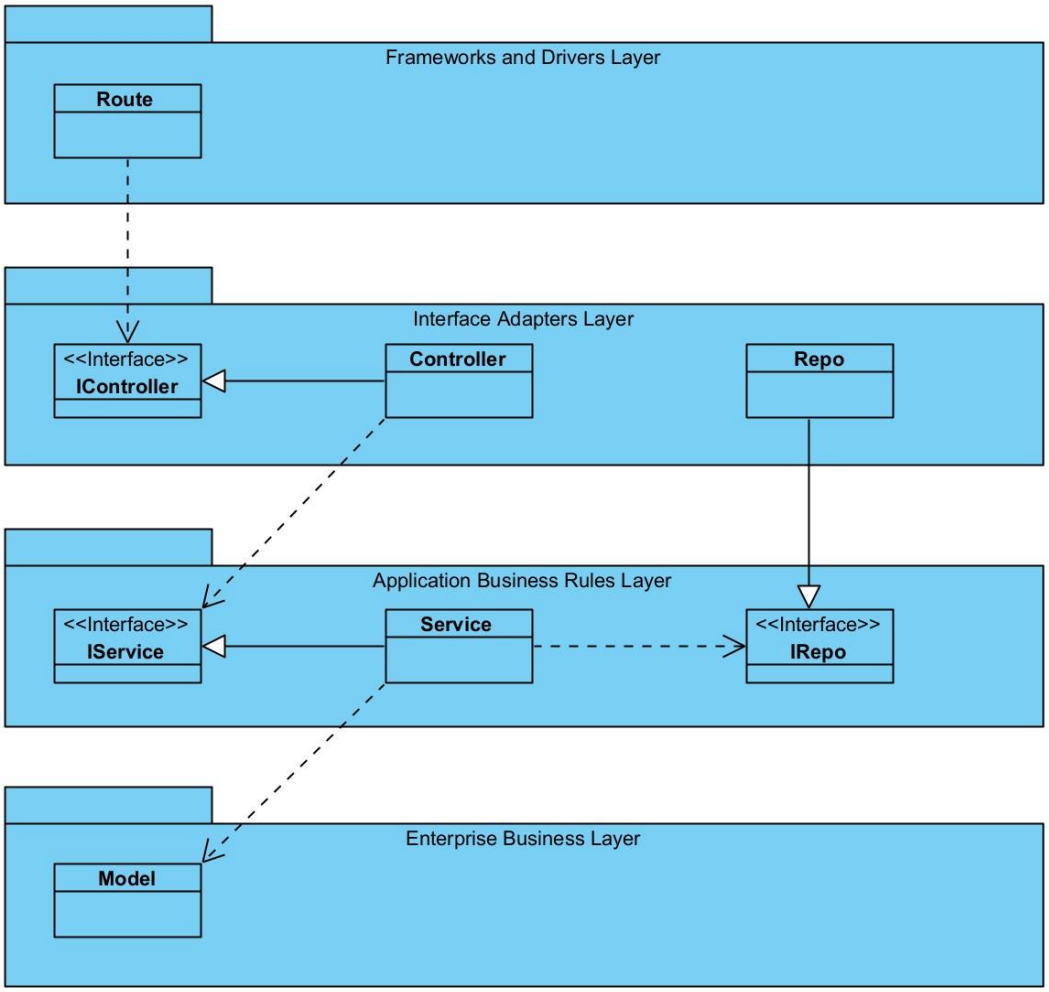

Figura 3.5: Vista de implementação

Nesta vista evidenciam-se camadas com dependências orientadas do exterior para o interior que terminam com interfaces. Esta em conformidade com o Dependency Inversion Principle (DIP). Assim, camadas internas não referenciam diretamente implementações externas, por exemplo: Service depende de IRepo e não de Repo.

A vista seguinte incide sobre o componente *front-end* da solução.

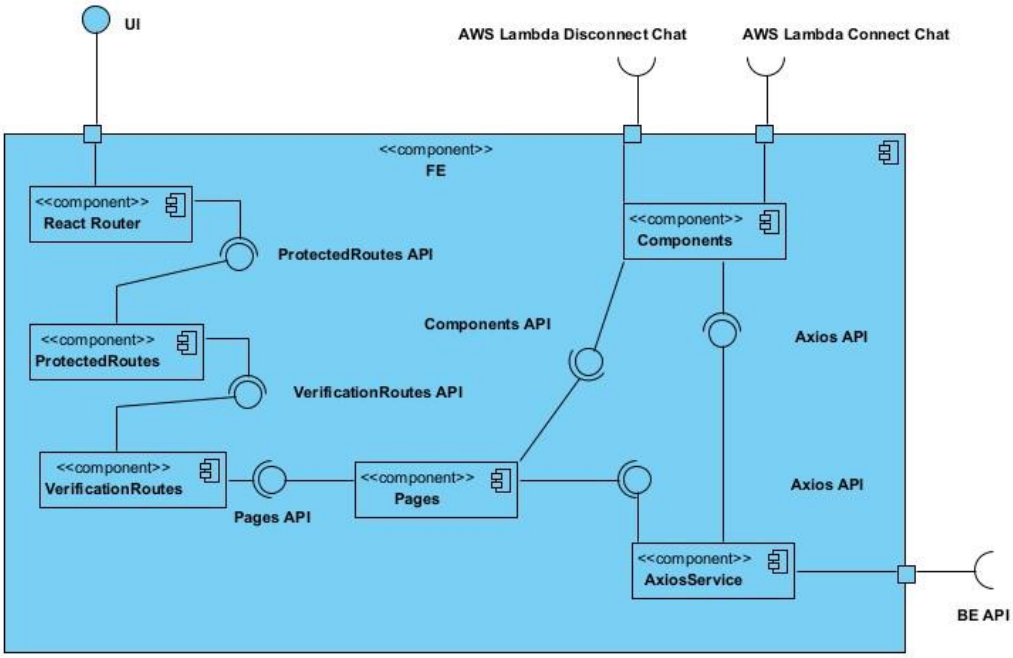

Figura 3.6: Vista Lógica Nível 3 - *Front-End*

A Figura 3.6 apresenta os seguintes componentes:

- React Router - Gere o sistema de navegação da aplicação
- Protected Route - verifica as permissões de acesso, no caso da solução verifica se o utilizador está autenticado.
- Verification Route - Aumenta a barreira de segurança, pois verifica se o utilizador tem permissão para assistir ou criar uma transmissão.
- Pages - Conjunto das páginas da aplicação.
- Components - Biblioteca de componentes reutilizáveis pelas páginas ou por outros componentes.
- Axios Service - Responsável pela comunicação com o *back-end*

3.4.5 Vista Física

Como toda a infraestrutura da solução reside na Amazon Web Services (AWS), nesta fase de desenho foi necessário aplicar os conhecimentos obtidos na formação.

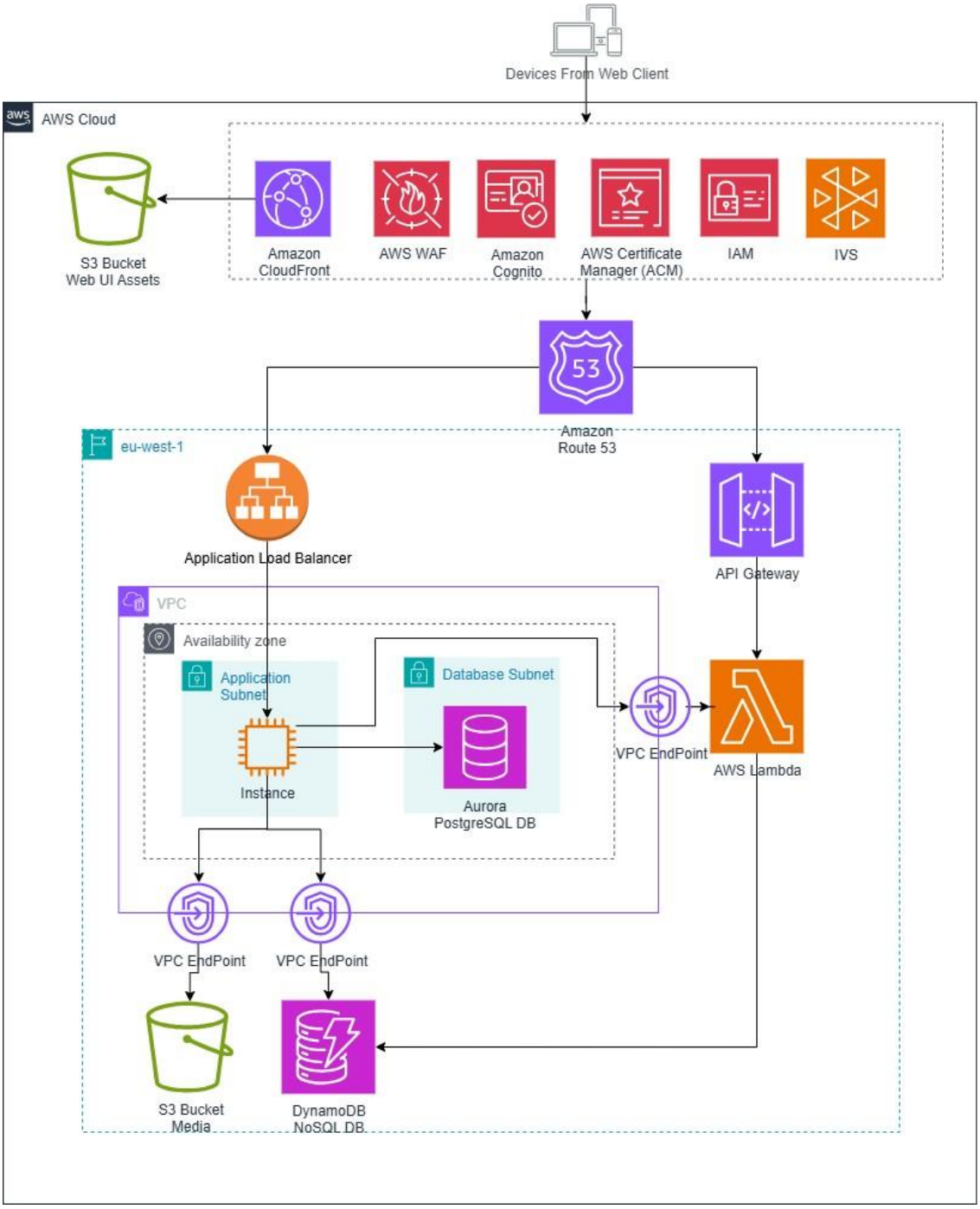

Figura 3.7: Vista física - Infraestrutura AWS

Na Figura 3.7 são apresentados os seguintes serviços:

Tabela 3.6: Serviços AWS e respetivo papel na solução

Serviço	Descrição
Amazon Route 53	Resolução de nomes do domínio da aplicação
Amazon CloudFront	Distribuição de conteúdos estáticos do S3
Amazon S3 (UI)	Contêm os conteúdos estáticos do *front-end*
Amazon S3 (Media)	Repositório de conteúdos multimédia (vídeos/fotos) .

AWS Web Application Firewall (WAF)	Controlo do tráfego na aplicação
AWS Certificate Manager (ACM)	Emite certificados Secure Sockets Layer (SSL) para a aplicação.
Application Load Balancer (ALB)	Distribui o tráfego de pedidos para diversas instâncias da aplicação.
Amazon Virtual Private Cloud (VPC)	Isola a rede da aplicação com, *subnets*, *security groups* e *routing table*.
VPC Endpoints	Permite aceder a serviços fora da VPC.
Amazon API Gateway	Expõem *enpoints* para o acesso as funções Lambda.
AWS Lambda	Função *serverless* que consegue fazer *build* de codigo independente.
Amazon Aurora PostgreSQL	Base de dados SQL
Amazon DynamoDB	Base de dados NoSQL
AWS IVS	Serviço gerido pela AWS que cria os canais de transmissão e as credenciais únicas de cada criador.
AWS Identity and Access Management (IAM)	Controla o acesso da aplicação.

Passando para uma explicação mais estruturada, em primeira instância, os dispositivos dos utilizadores acedem à aplicação, cujo nome é resolvido no Amazon Route 53.

O conteúdo do *front-ent* é alojados pelo Amazon CloudFront a partir de Amazon S3, com proteção AWS WAF e para segurança através do protocolo Transport Layer Security (TLS) via Amazon Certificate Manager.

As chamadas do *front-end* seguem dois percursos:

- Através de um Application Load Balancer para a APIREST do *back-end*.
- Através *API Gateway* para funções *AWS Lambda* quando adequado.

A persistência é feita por Amazon Aurora PostgreSQL (base de dados SQL), DynamoDB (base de dados NoSQL) e S3 (para objetos como imagens e vídeos).

O AWS IVS suporta as transmissões em direto e o IAM controla o acesso aos recursos da infraestrutura.

Com estes serviços a solução garante escalabilidade, disponibilidade e segurança.

3.4.6 Modelo de Dados

Após a descrição do sistema, o modelo de dados foi desenhado para ajudar na compreensão do domínio e na construção das bases de dados.

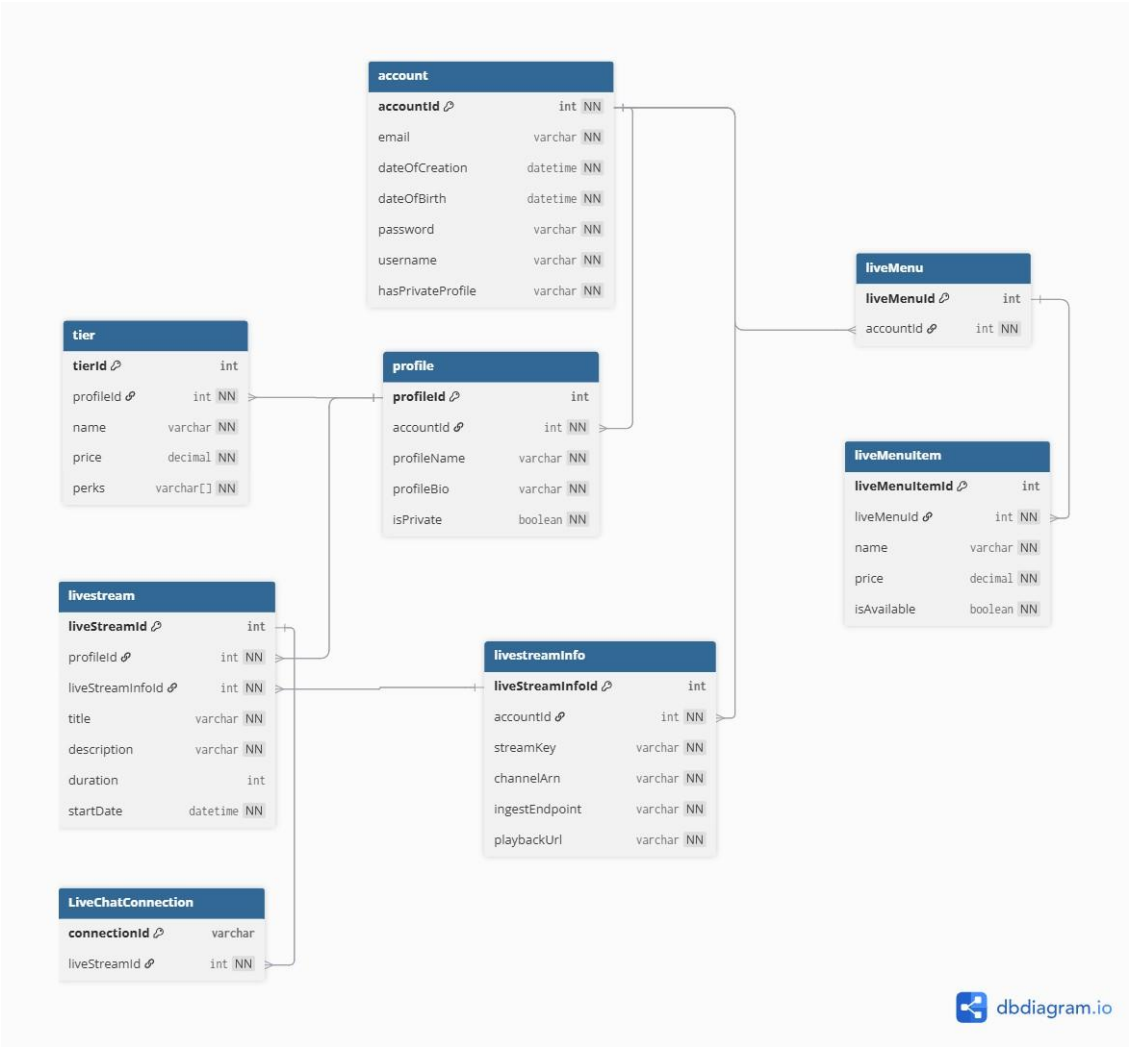

Figura 3.8: Modelo de Dados

Na Figura 3.8, está representado o modelo de dados adotado para a criação do dominio do código.

Adotou-se um modelo híbrido, que combina a base de dados relacional (PostgreSQL) e a base de dados não-relacional (DynamoDB).

A camada relacional preserva a integridade do modelo, onde são garantidas as regras de negocio e transações Atomicity, Consistency, Isolation, Durability (ACID).

A camada não-relacional serve para objetos de grande leitura e editáveis.

SQL

- Account: accountId, email, dateOfCreation, dateOfBirth, password, username, hasPrivateProfile.
- Profile: profileId, accountId, isPrivate.
- Tier: tierId, profileId.

- LiveStreamInfo: liveStreamInfoId, accountId, streamKey, channelArn, ingestEndpoint, playbackUrl.
- LiveStream: liveStreamId, profileId, liveStreamInfoId.
- LiveMenu: accountId, liveMenuId.
- LiveMenuItem: liveMenuItemId, liveMenuId, isAvailable.

DynamoDB

- Profile: profileId, name, bio.
- Tier: tierId, name, price, perks.
- LiveStream: liveStreamid, title, description, duration, startDate.
- LiveMenuItem: liveMenuItemId, name, price.
- LiveChatConnection : connectionId, liveStreamId.

Como as transmissões não serão gravadas, não houve a necessidade de fazer um modelo de dados para o *chat*.

3.4.7 Diagramas de sequência dos casos de uso

Nesta secção apresentam-se diagramas que explicam como cada caso de uso é executado ao nível das interações entre o ator e o sistema.

Para privilegiar a legibilidade e reduzir a poluição visual, os diagramas incluídos no corpo do texto encontram-se deliberadamente simplificados, representando apenas dois participantes: *Ator* e *Sistema*. O participante *Sistema* agrega, de forma abstrata, os componentes internos e respetivos fluxos.

As versões completas e detalhadas dos diagramas estão na secção dos anexos:

- Diagrama de Sequência do *back-end* A.6.
- Diagrama de Sequência do *front-end* A.5.
- Diagrama de Sequência do sistema A.4.

Casos de Uso C1 & C2

O diagrama de sequência 3.9, cobre os casos de uso que descrevem a configuração inicial do criador para criar um ambiente de transmissão. Descreve as validações necessárias para criar o título e a descrição, e inclui a persistência de dados.

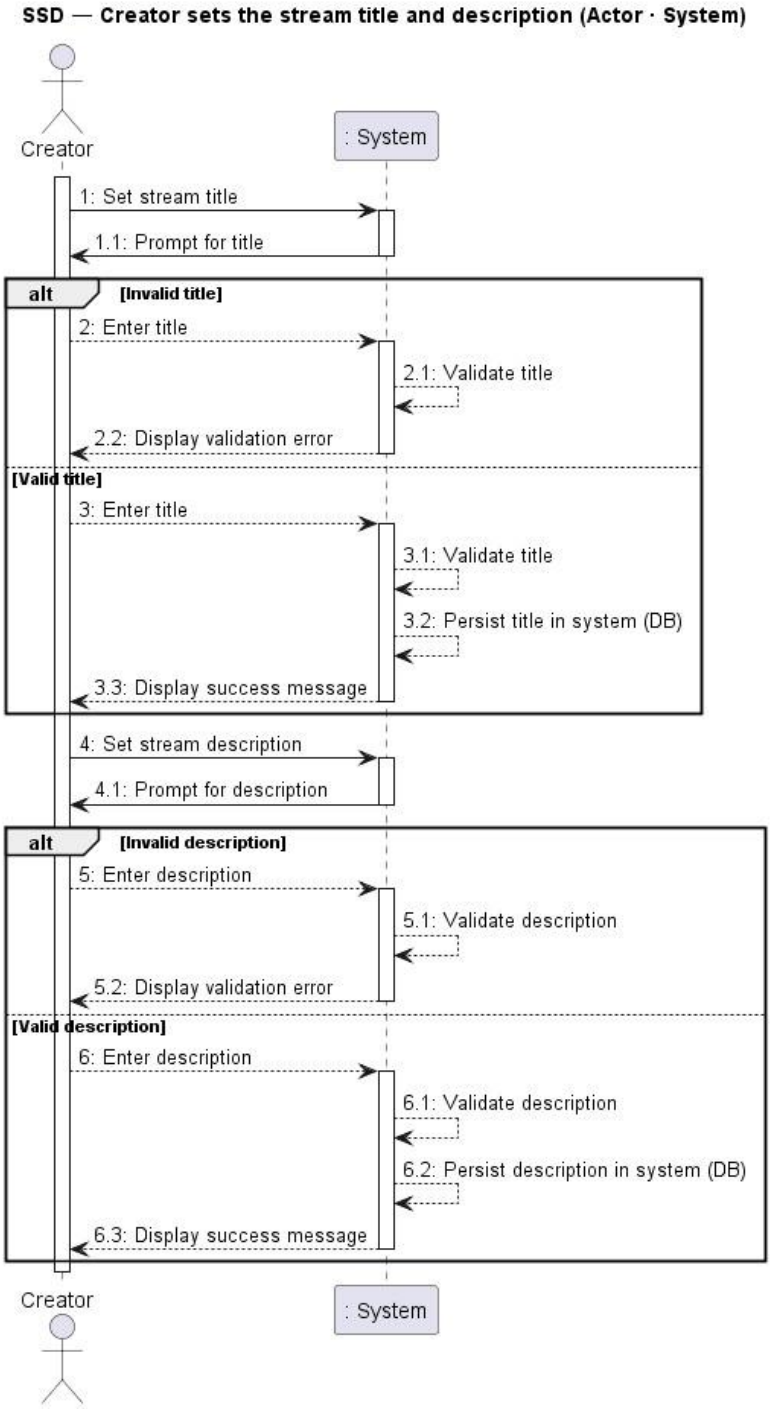

Figura 3.9: Diagrama de sequência do sistema para criar título e descrição da transmissão.

Casos de Uso C6 & C7 & C3

O diagrama de sequência 3.10, mostra a configuração da câmara e do microfone, através do *MediaStream* referido anteriormente no capítulo 2, com permissões pedidas pelo *browser*, ao utilizador e uma *preview* da transmissão.

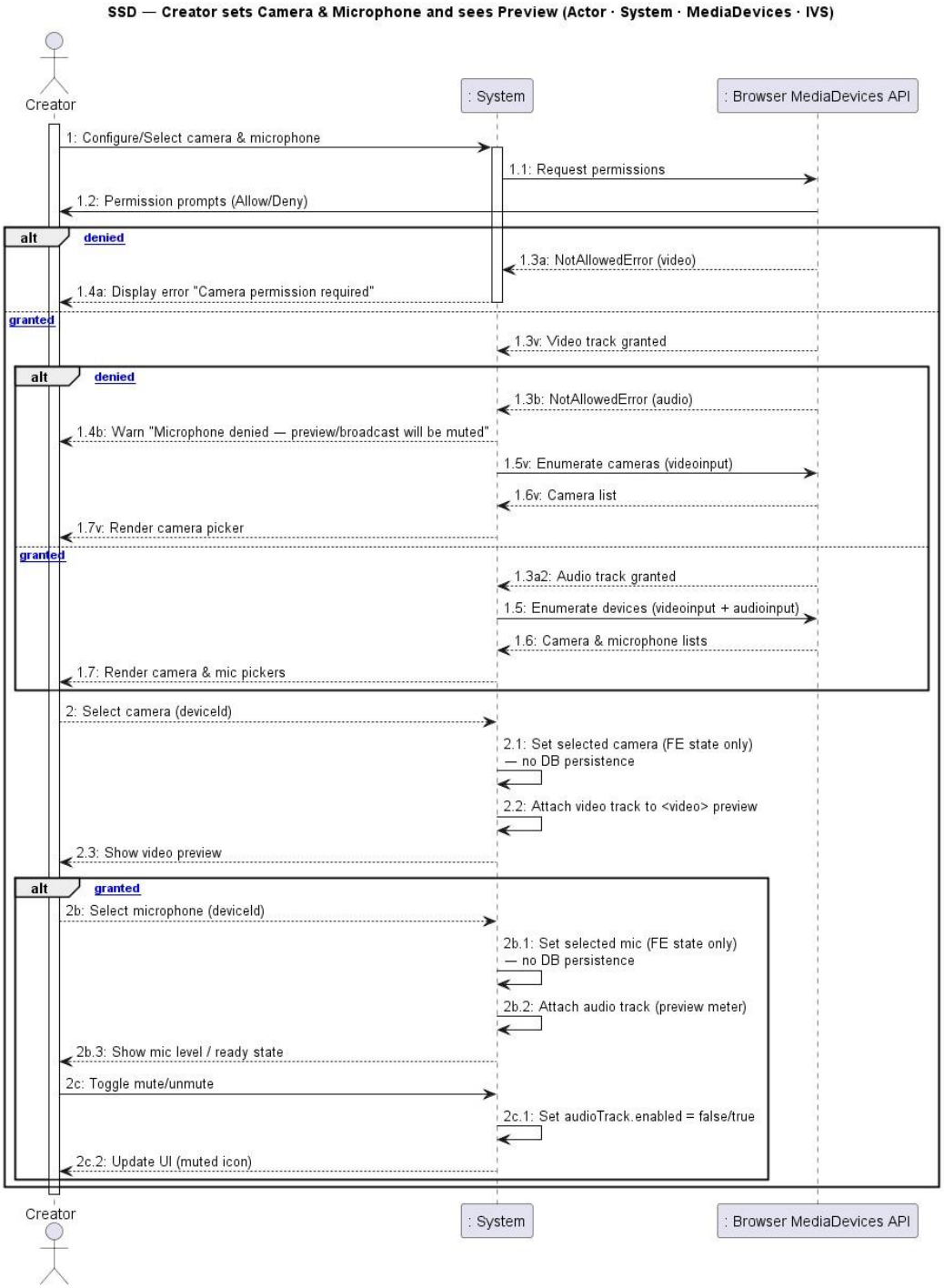

Figura 3.10: Diagrama de sequência do sistema para configurar câmara e microfone (ativação, permissões e teste).

Casos de Uso C8 & C9

O diagrama de sequência 3.11, descreve a ação de desativar e reativar o microfone e desligar a câmara. Ambas as ações geram notificações ao utilizador melhorando a sua experiência.

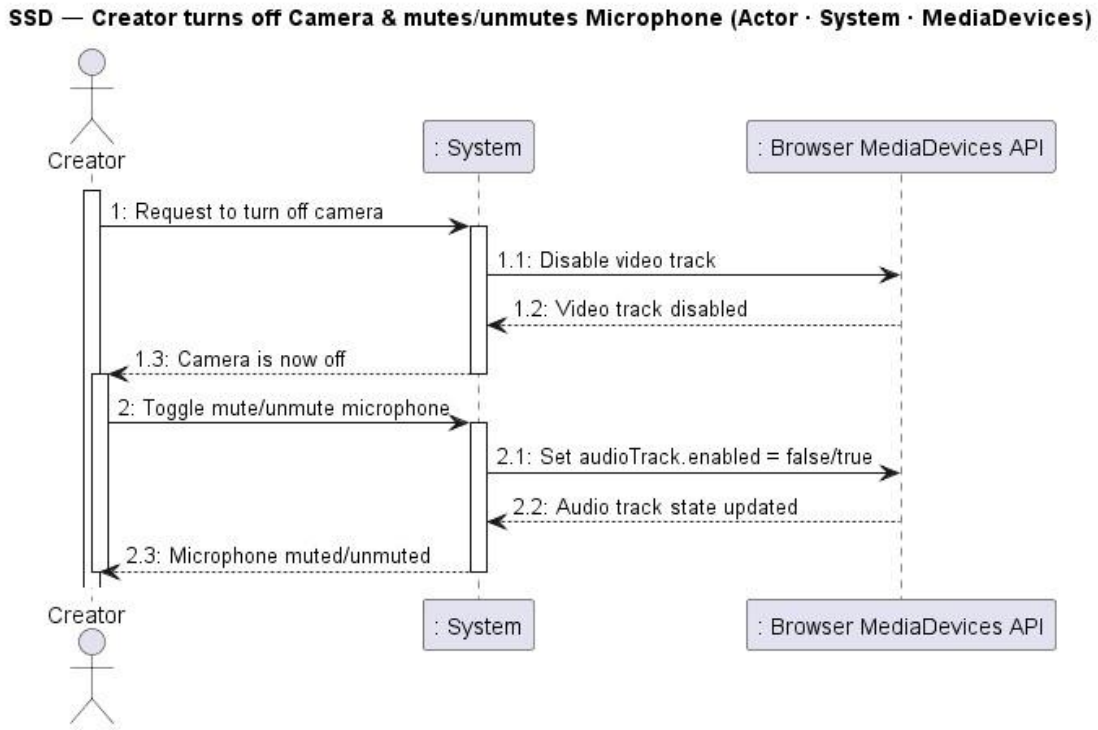

Figura 3.11: Diagrama de sequência do sistema para desativar/reativar câmara e microfone durante a transmissão.

Casos de Uso C5 & C10

O diagrama de sequência 3.12, simplifica e descreve a ação de começar uma transmissão, o sistema verifica todos os requisitos antes de criar a *stream*.

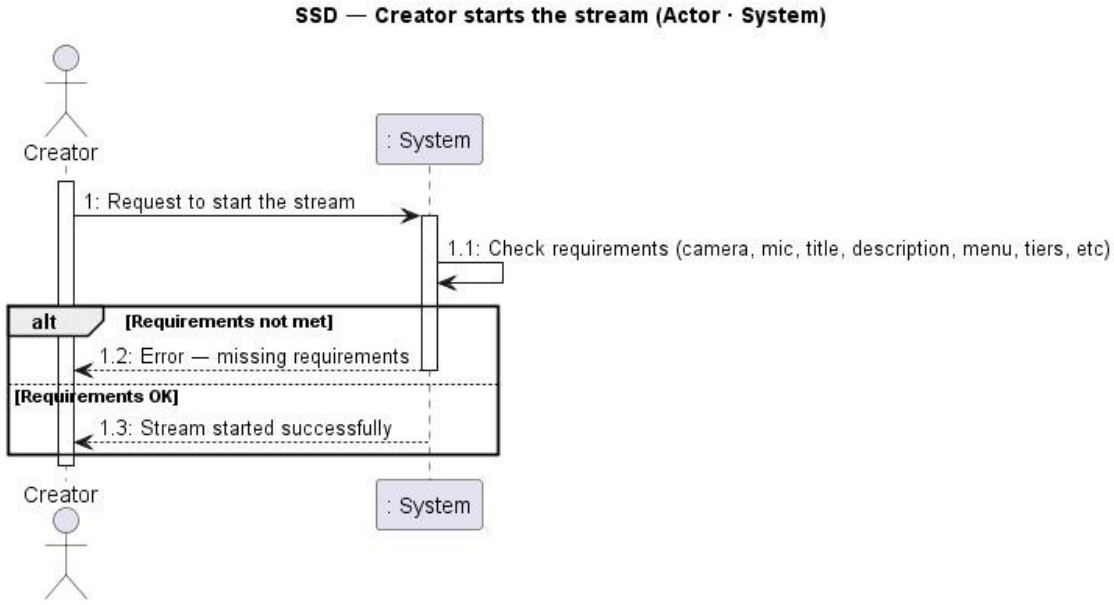

Figura 3.12: Diagrama de sequência do sistema para iniciar a transmissão (*start stream*).

O diagrama de sequência 3.13, descreve a simplicidade de acabar uma transmissão.

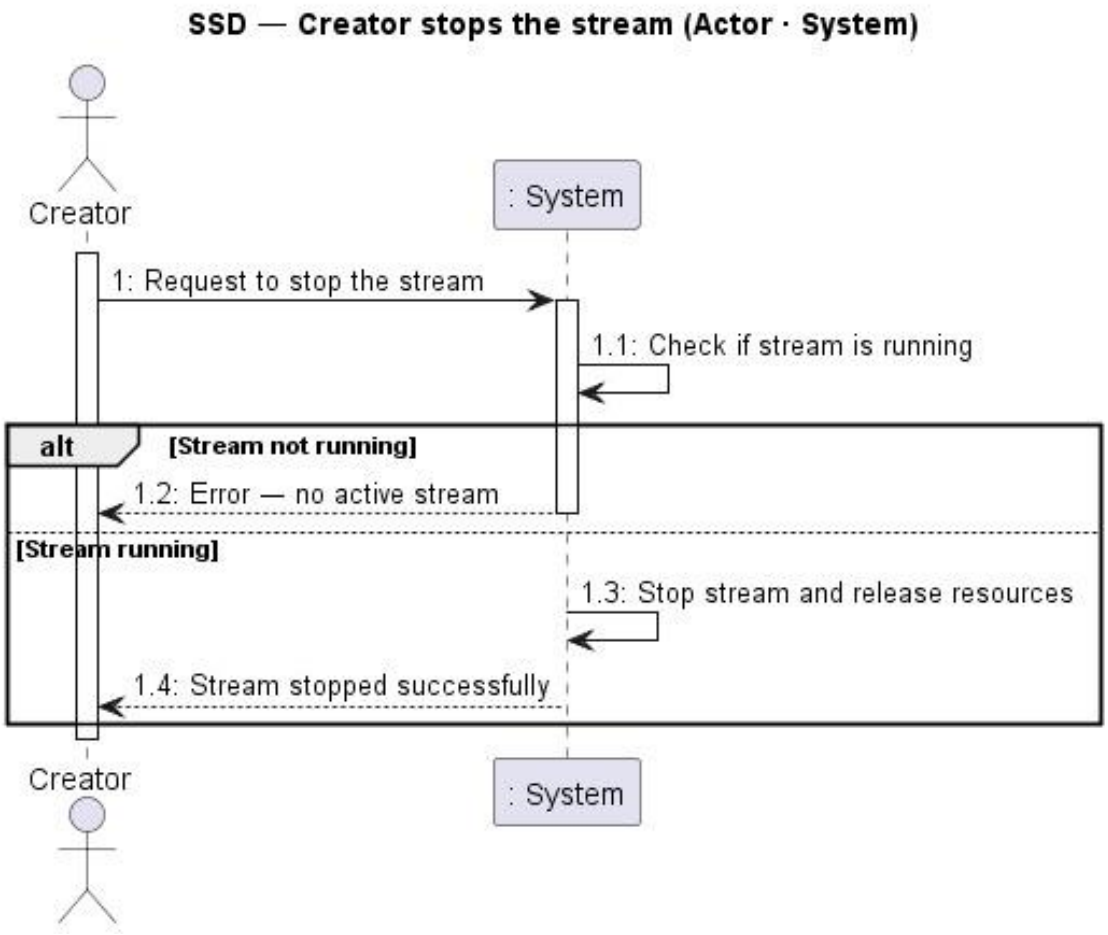

Figura 3.13: Diagrama de sequência do sistema para terminar a transmissão (*stop stream*).

Caso de Uso C11

O diagrama de sequência 3.14, descreve como o criador entra na sua própria página de *stream*, e por *web-socket*, liga-se a sala de *chat*. Após ligar-se fica capaz de receber mensagens em tempo real por outros utilizadores que estejam na mesma sala.

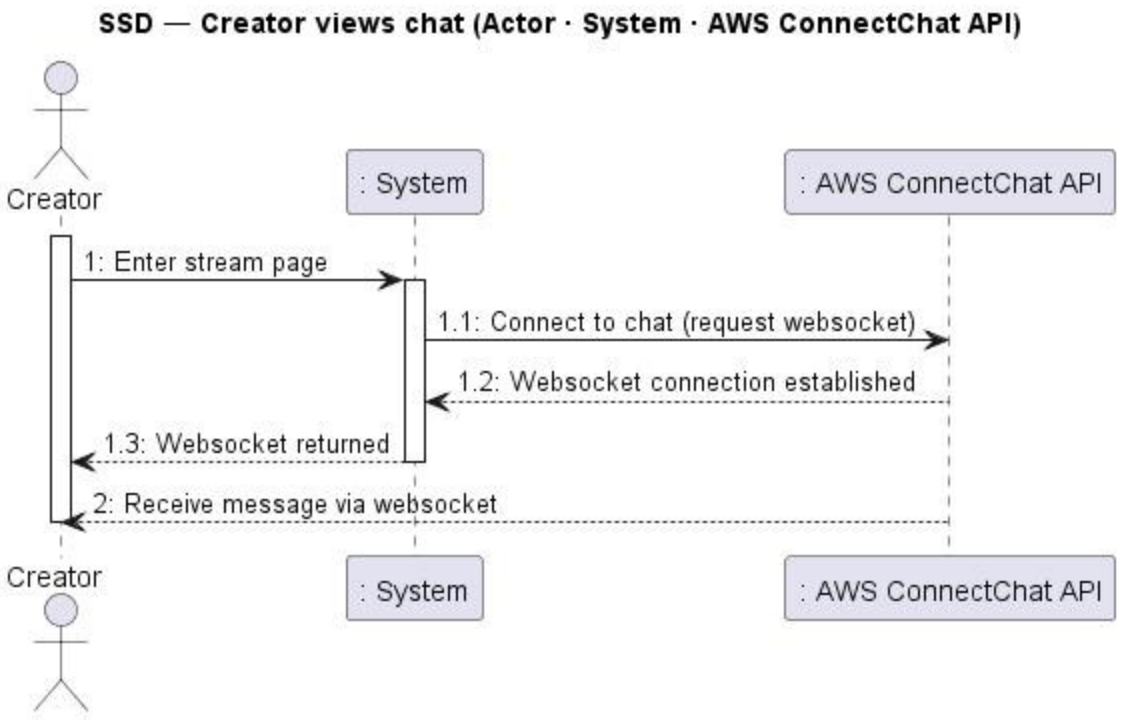

Figura 3.14: Diagrama de sequência do sistema para visualização do chat pelo criador.

Caso de Uso C12

O diagrama de sequência 3.15 especifica a interação ativa do criador com o chat: envio de mensagens, respostas, remoção e moderação. Detalham-se confirmações de entrega (*ack*), tratamento de *rate limits*, e feedback visual para estados em envio/erro. Prevê-se ainda a sincronização com serviços anti-spam e logs de auditoria.

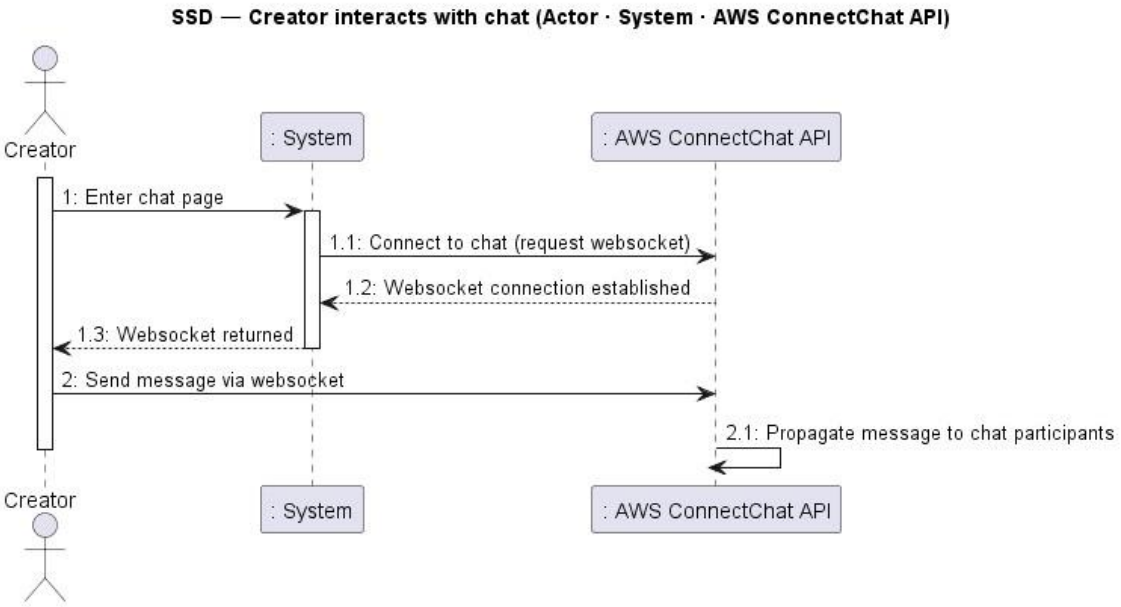

Figura 3.15: Diagrama de sequência do sistema para interação do criador no chat (envio e moderação).

Caso de Uso C13

O diagrama de sequência 3.16 foca a gestão e apresentação do *Live Menu* durante a transmissão. Descrevem-se a consulta do catálogo, a atualização de disponibilidade em tempo real e a reflexão imediata das alterações na interface. São também contemplados eventos de sincronização com o backend e métricas de uso.

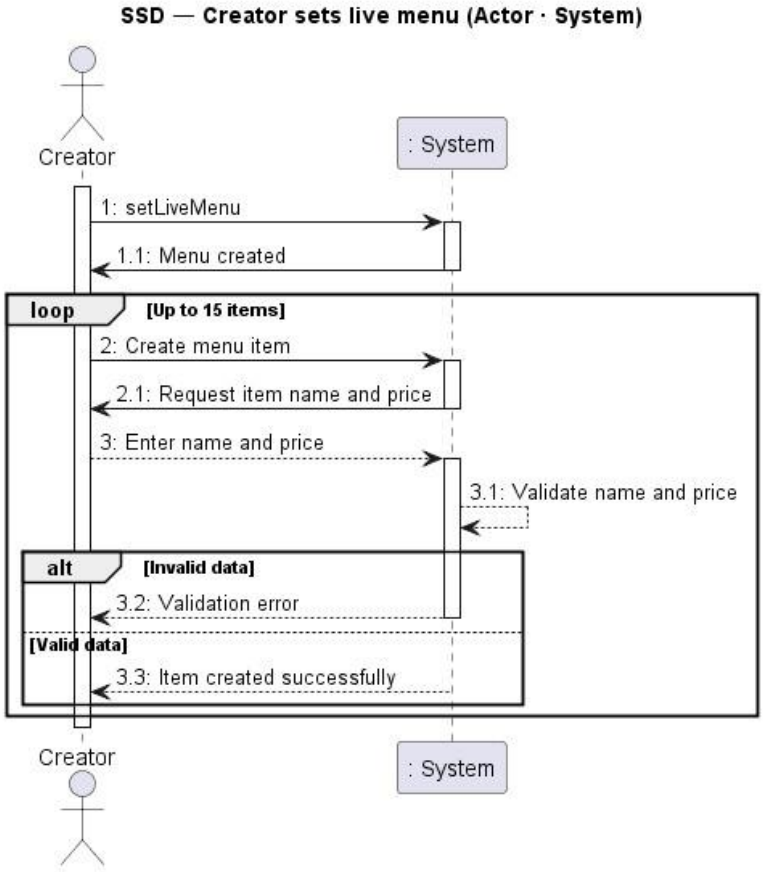

Figura 3.16: Diagrama de sequência do sistema para gestão e apresentação do *Live Menu*.

Caso de Uso C14

No diagrama de sequência 3.17, aborda-se a gestão de *tiers* de acesso e a sua seleção/atribuição à sessão.

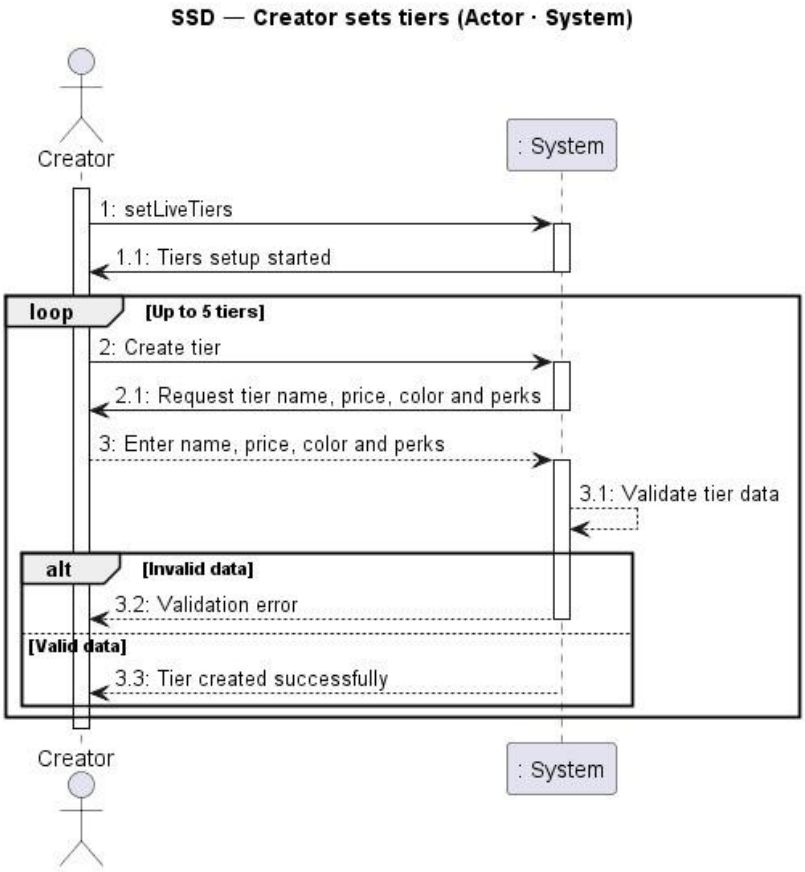

Figura 3.17: Diagrama de sequência do sistema para gestão e seleção de *tiers* de acesso.

Caso de Uso V1

O diagrama de sequência 3.18 descreve a experiência do *viewer* ao iniciar a visualização duma transmissão. Incluem-se a resolução de URL do canal, o estabelecimento da sessão de reprodução, a negociação de qualidade adaptativa e o tratamento de erros iniciais.

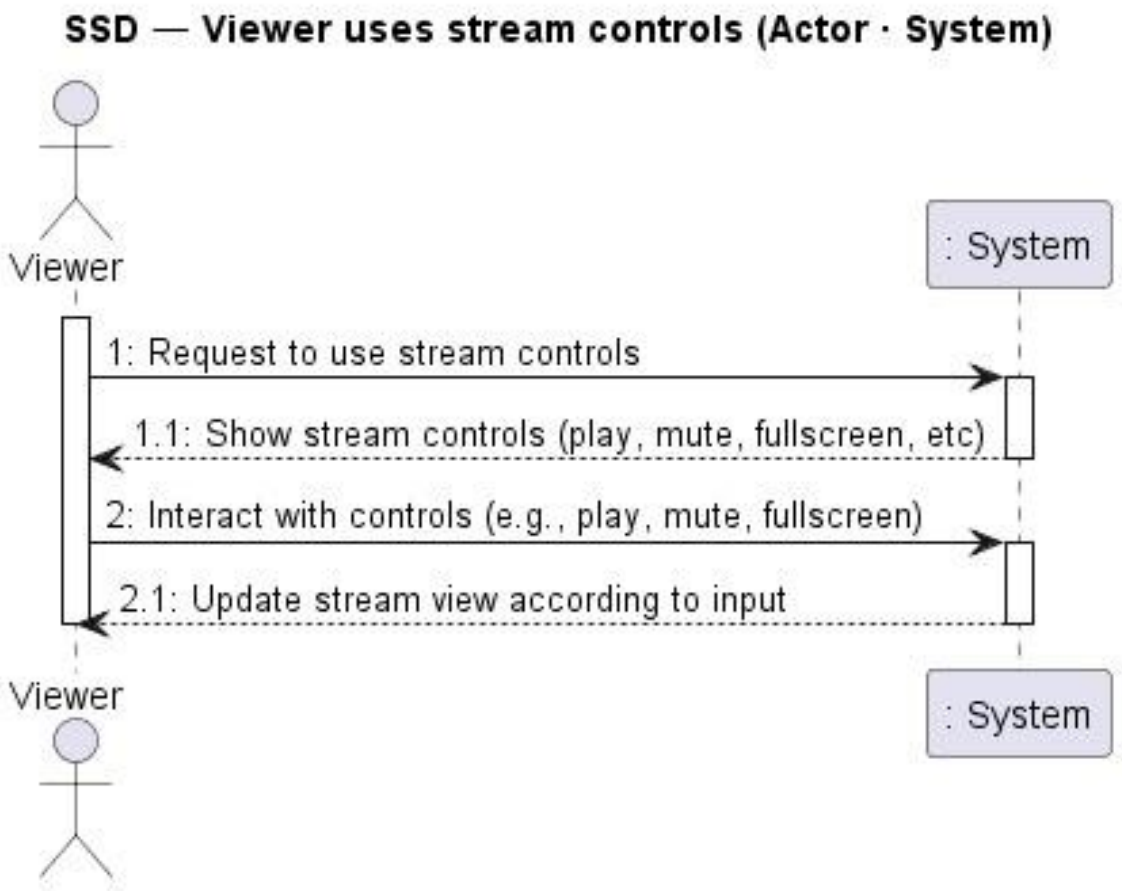

Figura 3.18: Diagrama de sequência do sistema para visualização da transmissão pelo utilizador.

Caso de Uso V2

O diagrama de sequência 3.19 aborda a interação do utilizador com o *chat* público/privado durante a transmissão. Explicita-se o envio e receção de mensagens em tempo real, limitações de frequência, e *feedback* de entrega. Prevê-se moderação automática e mecanismos de denúncia.

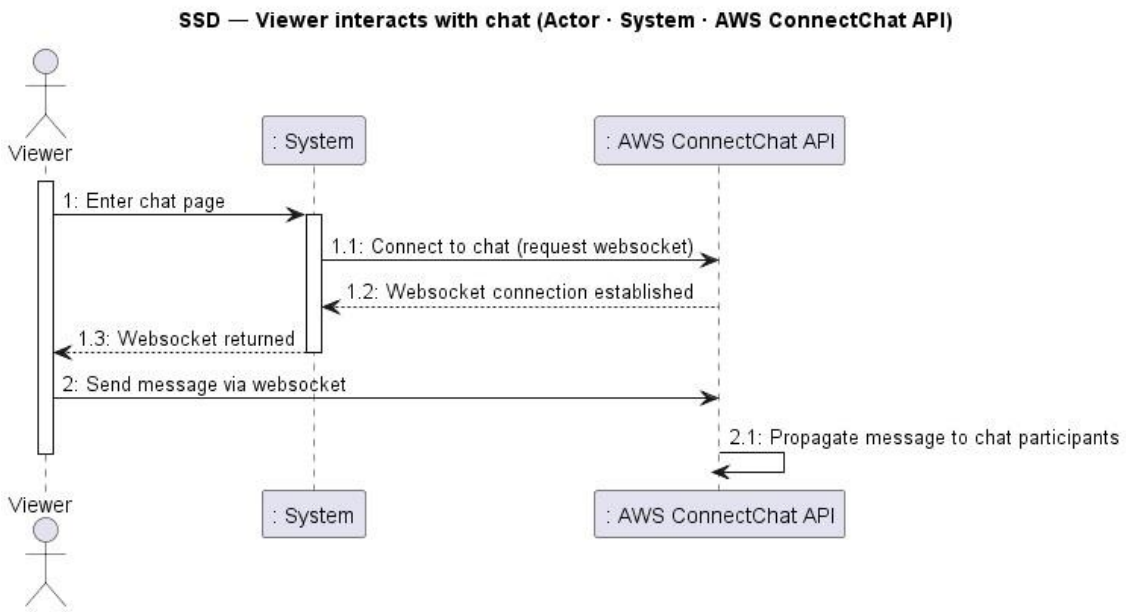

Figura 3.19: Diagrama de sequência do sistema para interação do utilizador no chat.

Caso de Uso V3

No diagrama de sequência 3.20 detalham-se os controlos de reprodução disponíveis para o utilizador: *play/pause*, volume, *mute*, ecrã inteiro e seleção de qualidade. Define-se a sincronização de estado entre UI e *player*, incluindo atalhos e acessibilidade. Consideram-se ainda indicadores de estado (buffering, erro) e mensagens de orientação.

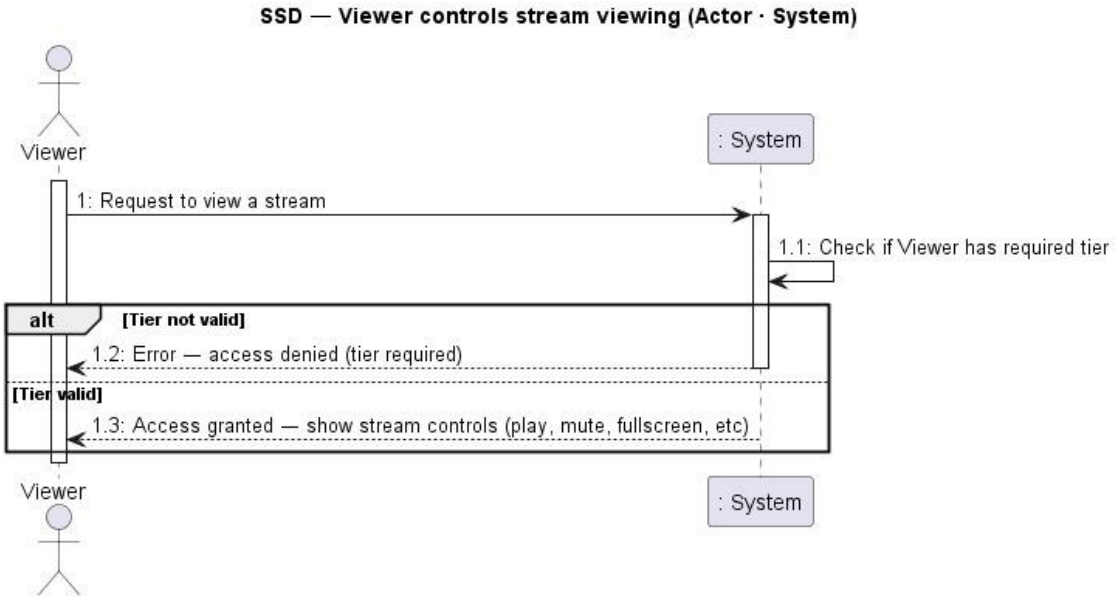

Figura 3.20: Diagrama de sequência do sistema para controlos de reprodução da transmissão pelo utilizador.

3.4.8Alternativas ao design

Consoante o desenvolvimento, foram detetadas alternativas ao desenho da solução, nomeadamente na criação da transmissão. Nos caso de uso apresentados, as configurações como o título de a descrição são persistidos na base de dados antes de iniciar a *live*, por outro lado, poderia ser adequado só persistir nas bases de dado, apenas quando fosse começar uma transmissão.

Em alternativa ao modelo de como as salas de transmissões são feitas, seria prudente criar uma *web-socket*, para a sala em geral, pois para interações em tempo real, não seria necessário criar uma ligação nova a para cada funcionalidade. Exemplificando, quando um visualizador que doar ao criador durante a transmissão, precisa de ser despoletado uma ação de confirmação, e como esse serviço não está ligado a nenhuma "sala", não consegue ser em tempo real. Para resolver este caso foi desenvolvido um "scheduler"no *back-end*, que corre certos métodos em curtos espaços de tempo.

3.5Conclusão

O capítulo apresentou uma visão completa e coerente do sistema, desde a caracterização do domínio e definição de requisitos até à arquitetura e modelos que os materializam. Clarificaram-se conceitos nucleares (*Account*, *Profile*, *Tier*, *LiveStream*) e respetivas relações, assegurando uma base semântica estável para o desenho técnico. Foram especificados requisitos funcionais (RF01–RF08) e não funcionais, enquadrados no modelo FURPS+, garantindo alinhamento entre necessidades de negócio, usabilidade, fiabilidade, desempenho e suportabilidade.

O desenho arquitetural, estruturado pelo modelo *C4*, evidenciou a separação *FE/BE*, responsabilidades por camadas e dependências internas, culminando numa vista física detalhada em *AWS*

O modelo de dados híbrido (relacional + NoSQL) foi alinhado com o domínio: integridade transacional em Aurora para entidades críticas e projeções e editabilidade em DynamoDB para leitura intensiva, reforçando tanto consistência como performance. Por fim, os diagramas de sequência consolidaram o comportamento por casos de uso.

Apêndice A

Diagrama de casos de uso e Diagramas de sequencia

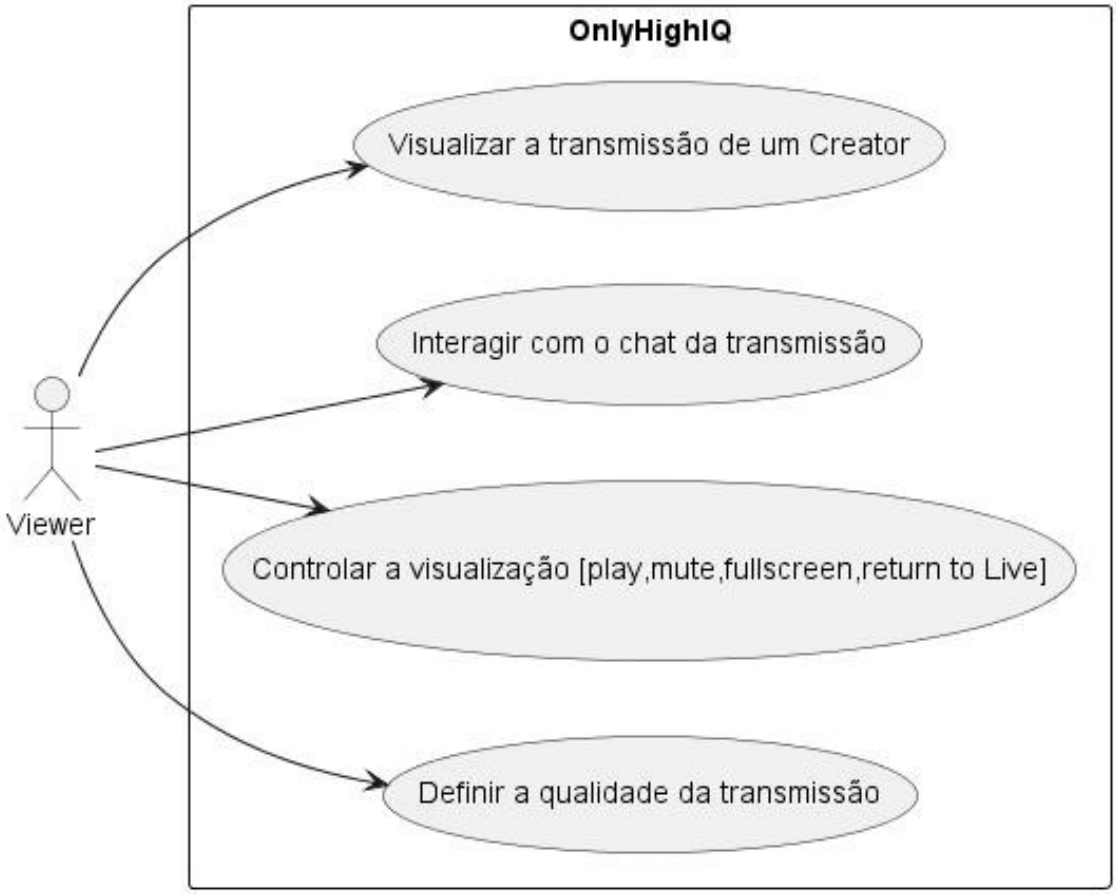

Figura A.1: Diagrama de casos de uso - Viewer

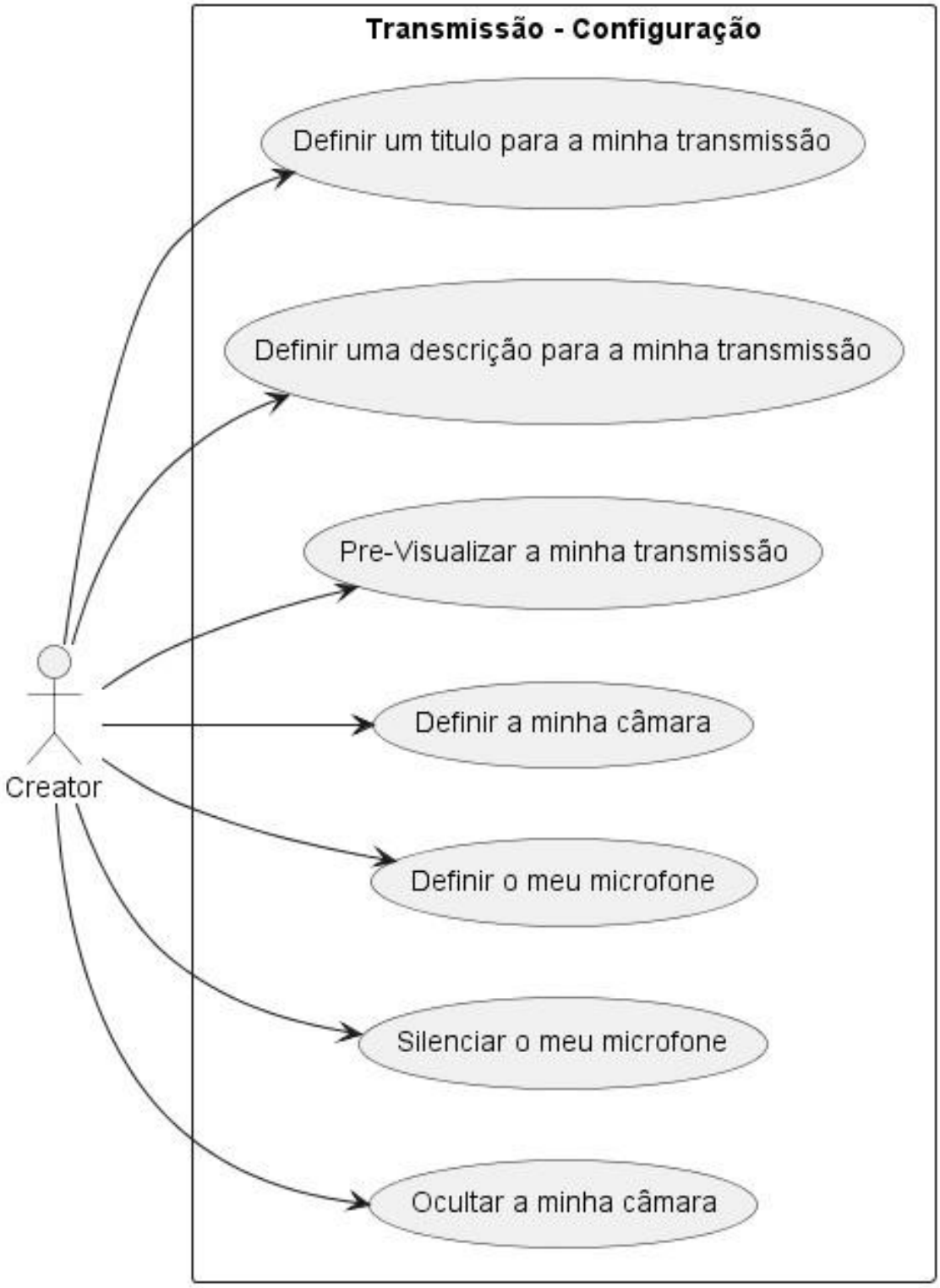

Figura A.2: Diagrama de casos de uso - Creator Configuração

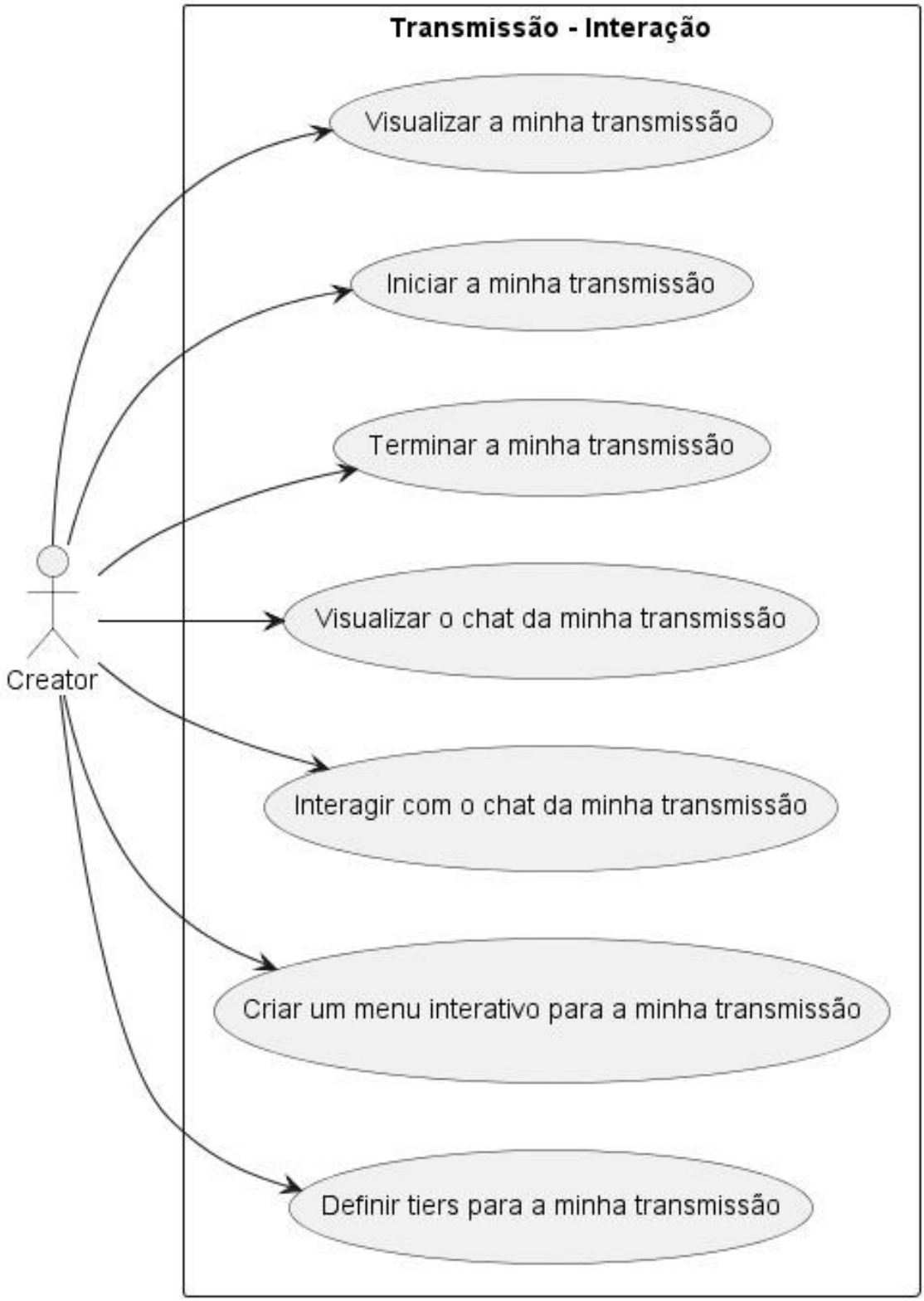

Figura A.3: Diagrama de casos de uso - Creator Interação

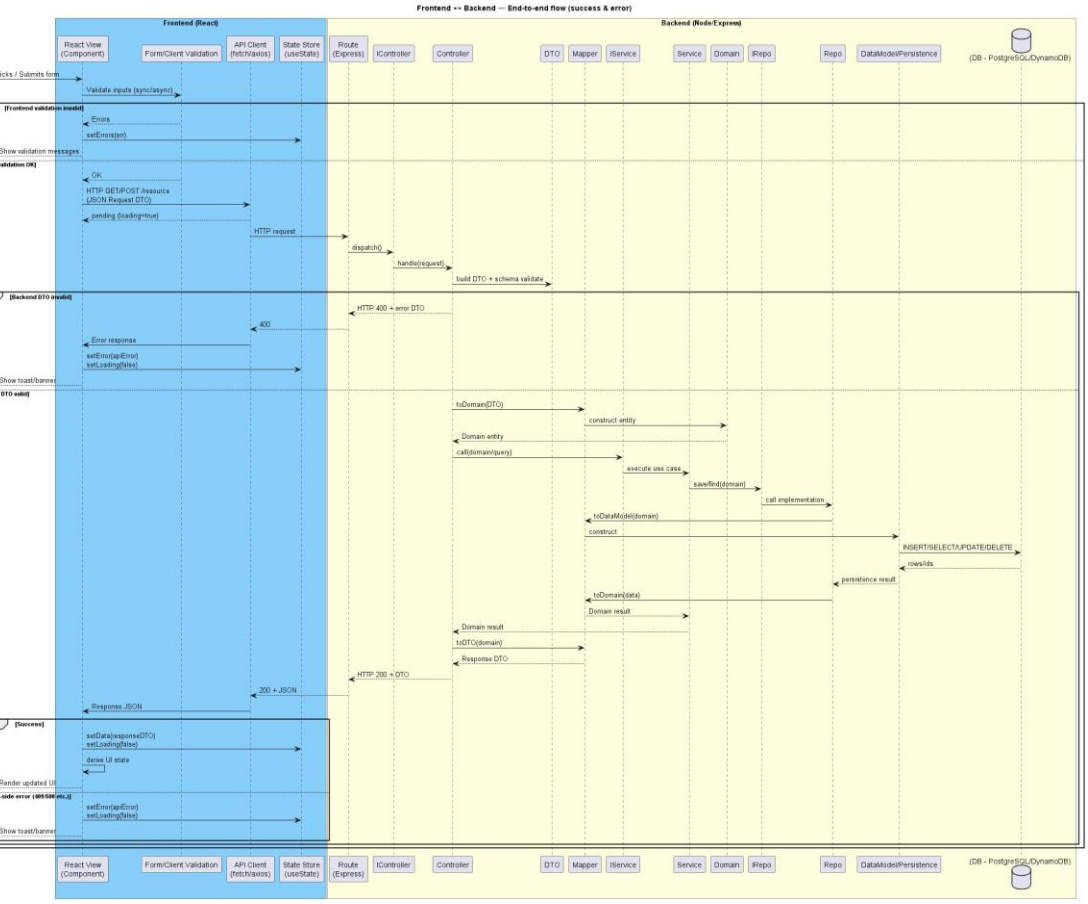

Figura A.4: Diagrama de Sequência completo

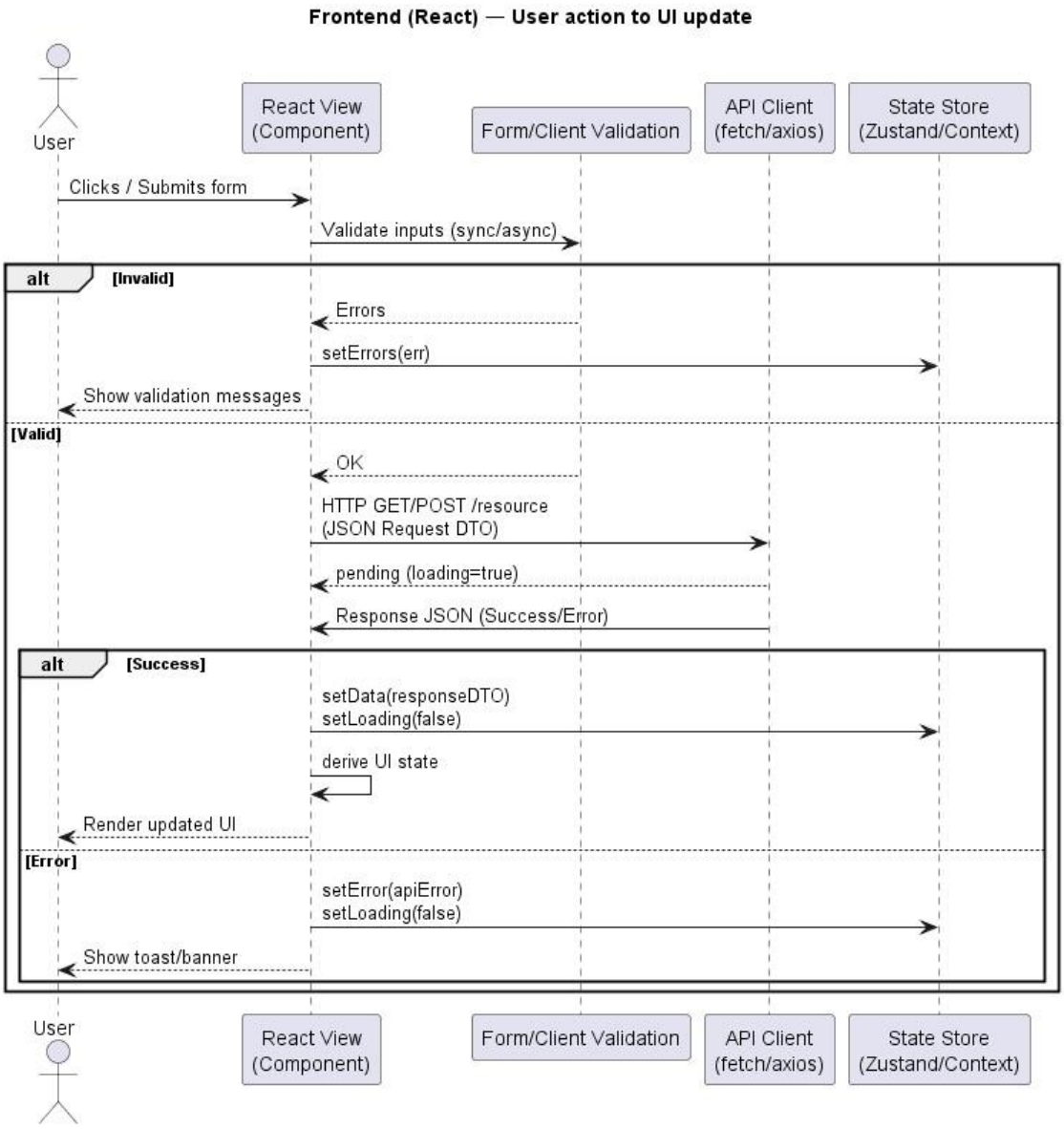

Figura A.5: Diagrama de Sequência - *Front-End*

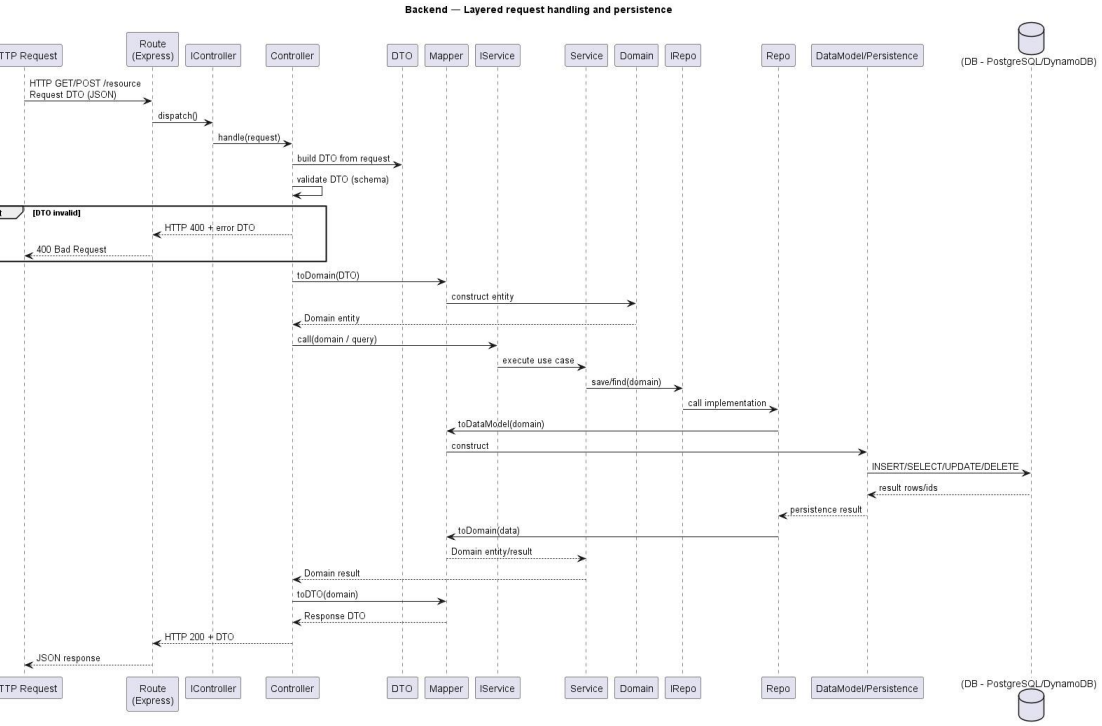

Figura A.6: Diagrama de Sequência - *Back-End*